
Experimental Design Analysis of a Rocket Test Stand with Pneumatic Braking

Engineering Experimentation [ME 360]
Prof. Sidebotham, Prof. Wootton, Prof. Lawless
15 May 2026
Jake Peyser, David Brokhin, and Sri Senthil

Abstract

An experimental air rocket test stand was developed to measure rocket position and acceleration in a controlled and repeatable environment. The system serves as an educational platform for studying fundamental principles in fluid mechanics, thermodynamics, and dynamics. The apparatus consists of a base launch mechanism that is integrated with a 2.3-meter-tall vertical guide tube with a 10-centimeter internal diameter, which acts to constrain the rocket's motion to one dimension, simplifying analysis. A vent near the top of the tube restricts airflow ahead of the rocket, producing a pneumatic braking effect that limits peak velocity and improves repeatability. The rocket itself is constructed from a 1- or 1.5-liter plastic bottle with a nozzle that is pressurized prior to release and is propelled by the expulsion of air. An onboard accelerometer is used to record the rocket's acceleration. By varying the initial pressure, volume, and rocket mass, the system allows for the comparison of experimental results and theoretical results for different initial conditions through quantitative analysis.

I. Introduction

Undergraduate engineering laboratories often rely on simplified demonstrations of fundamental concepts that can limit a student's ability to connect theoretical models to real-world system behavior. While these demonstrations are an effective means for demonstrating basic concepts, they lack the structure necessary for further rigorous analysis. Bottle rockets, for example, are commonly used to demonstrate principles of kinematics and energy. However, most of these demonstrations lack the ability to repeatedly collect reliable data to back up the concepts they seek to validate.

The objective of this project is to design and develop an experimental platform that provides a controlled, instrumented structure to study rocket motion through the launch, coasting and apogee phases. The system is intended to bridge the observed gap between theoretical learning and hands-on experimentation by providing a means to accurately measure the position and acceleration of the launch. While achieving this goal, the system should remain accessible and adaptable so that it can be used across a wide range of educational levels including laboratory research.

To meet these objectives, the system incorporates mechanical design and instrumentation. A vertical tube guides the rocket and constrains it to one-dimensional motion, while a pneumatic braking system limits the peak velocity of the rocket. An onboard accelerometer tracks the rocket's acceleration throughout the launch. These sensors along with the mechanical design allow for the acquisition of data to support a detailed analysis of the rocket's dynamics.

II. Experimental Design

i) Apparatus Components

The experimental apparatus consists of a base launch mechanism, transparent vertical guide tube, pneumatic braking system, a bottle rocket with a nozzle and nose cone, instrumented with necessary sensors for measuring position and acceleration. *Figure 1* shows this setup:

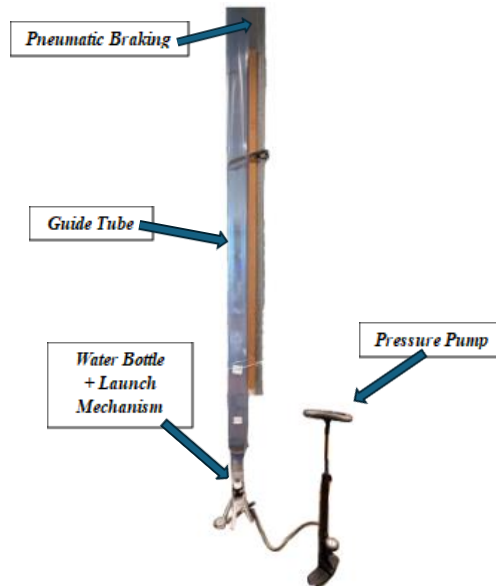


Figure 1 — Rocket apparatus including the vertical guiding launch tube, water bottle rocket, launch mechanism, pressure pump, and pneumatic braking system

The base serves as the launch platform, housing the release mechanism used to secure and pressurize the rocket before launch. The rocket is placed under the guide tube and released by pulling the tab on the launch mechanism when the desired initial pressure is reached. Because the base experiences a sudden large force when launched, a rigid base is necessary so that launch conditions remain consistent across all trials.



Figure 2 — Base launch mechanism used to secure, pressurize, then release the rocket

The guide tube is a 2.3-meter transparent vertical tube with a 10-centimeter internal diameter. The primary purpose of this tube is to constrain the rocket's motion to one-dimension. This simplifies the analysis by reducing the problem to position, velocity, and acceleration, all in the vertical direction. The transparency of the tube also allows for the motion of the rocket to be observed during testing, as well as allowing for the possibility of video analysis to be used as a secondary method if needed.



Figure 3 — The vertical guide tube restricts the rocket's motion to one-dimension and allows for visual observation and potentially the use of other methods to acquire relevant data.

The rocket is a standard 1- or 1.5-liter plastic bottle, fitted with a nozzle and nose cone. The bottle is where necessary launch pressure is built while the nozzle controls the outflow of air during the launch. The nozzle also experiences a buildup of pressure with an internal volume of 24.5-cm^3 when approximated as a cylinder with diameter 2.5-cm and depth of 5-cm. *Figure 4* shows the bottle with the pressure nozzle, as well as the instrumentation package both fitted on.



Figure 4 – 1.5-L water bottle rocket with the attached nozzles and instrumentation housing nose cone

The onboard electronics are fully enclosed in designed instrumentation housing. The nose cone features a 9-cm diameter to effectively contribute to the pneumatic braking effect, as well as an

internal design that allows for the stacking of the battery pack and the accelerometer, microcontroller, boost converter, pack. *Figure 5* shows some important characteristics and angles of the nose cone.

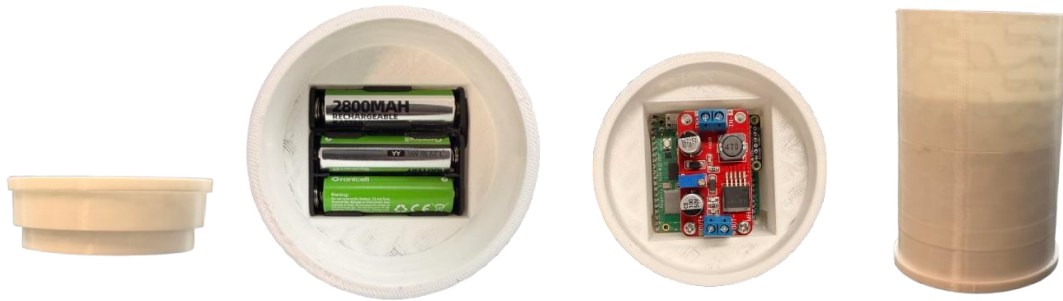


Figure 5 — From left to right, we see the nose cone cap, the battery housing, the accelerometer and other instrumentation housing, and the overall nose cone instrumentation package

Table 1 summarizes the function and relevant characteristics of the apparatus used to perform tests.

Component	Function	Relevant Specifications
Guide Tube	Constrains the rocket's motion to one-dimension and allows the launch to be observed visually while testing	2.3-m long transparent tube, positioned vertically; Contains a vent near the top for the pneumatic braking effect
Pneumatic Braking Attachment	An attachment to the water bottle that allows for the pneumatic braking effect to act significantly on the rocket	9-cm diameter; Attaches to the instrumentation housing and serves as a cap to cover instrumentation
Bike Tire Pump	Pressurizes the rocket prior to launch	100 psi maximum rating; 5 psi resolution
External Pressure Gage	Provides a more precise reading of pressure before launching	30 psi maximum rating
Water Bottle	Stores pressurized air before launch and is the base for instrumentation and pneumatic braking mechanisms.	Essentia 1 and 1.5-L bottle
Nose Cone	Houses the instrumentation package and attaches to the	Outer diameter of 9-cm
Bottle Coupler	Connects the water bottle to the pressurized chamber and launch mechanism	28 mm PCO thread profile; fits into pressurization chamber

i) Instrumentation Package Components

The rocket was instrumented to support the wireless acquisition of acceleration throughout the launch. The instrumentation package contains two accelerometers, a Raspberry Pi Pico W Microcontroller, a DC-DC boost converter, and rechargeable AA batteries. The microcontroller collects acceleration data from the sensors via the I²C and proceeds to transmit data wirelessly through a dual-band router to a computer for post-launch-processing.

Two accelerometers were used so that both lower and higher accelerations could be accurately captured. The Adafruit MPU6050 has acceleration ranges of ± 2 g, ± 4 g, ± 8 g, and ± 16 g and records along three-axes. This sensor is useful for the lower acceleration ranges in the coasting and apogee phases of the launch. At the start of the launch however, the rocket produces a large, impulsive acceleration, which cannot be read as precisely by this accelerometer. To address this limitation, the package also includes the Adafruit H3LIS331 accelerometer, which has acceleration ranges of ± 100 g, ± 200 g and, ± 400 g, to collect data for the launch instant. The two accelerometers together, as shown in *Figure 6* provide a more precise data set for the rocket.

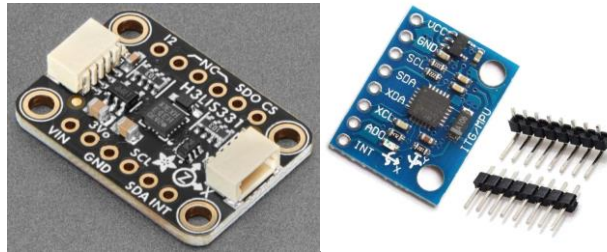


Figure 6 — Accelerometers used for onboard motion measurement, including the high-acceleration accelerometer (left) and low-acceleration accelerometer (right)

The accelerometers communicate with the Raspberry Pi Pico W Microcontroller (Pico) as seen in *Figure 7*. The microcontroller receives raw acceleration data and transmits the data wirelessly to a computer.

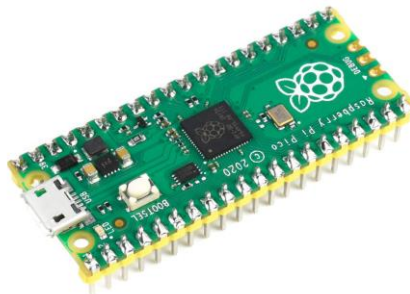


Figure 7 — Raspberry Pi Pico W microcontroller used to collect acceleration data from the onboard sensors, and transmit data wirelessly for processing

The Pico wirelessly transmits data to an AC1750 wireless dual-band router to send data to the computer for post-launch-processing.

The onboard electronics are powered by rechargeable AA batteries as shown in *Figure 9* and utilizes an ACEIRMC XL6019 DC-DC boost converter as shown in *Figure 8*, to provide the appropriate voltage of 5 V to the instrumentation. The boost converter maintains stable power onboard and limits voltage drops that may cause interruption in data collection.



Figure 8 — DC-DC boost converter used to regulate the power supply and provide the required voltage to the instrumentation



Figure 9 — Rechargeable AA battery pack used to provide power to the instrumentation onboard the rocket

Table 2 summarizes the function and relevant specifications of each of the discussed components used in the rocket’s instrumentation package.

Component	Function	Relevant Specifications
Adafruit MPU6050	Measures three-axis acceleration and angular motion during lower-acceleration testing	$\pm 2\text{g}$, $\pm 4\text{g}$, $\pm 8\text{g}$, $\pm 16\text{g}$ acceleration ranges with 16-bit resolution and I ² C communication
Adafruit H3LIS331	Measures three-axis acceleration during high-acceleration launch	$\pm 100\text{g}$, $\pm 200\text{g}$, $\pm 400\text{g}$ measurement ranges with 12 BIT measurement resolution and I ² C communication protocol
Raspberry Pi Pico W	Collects sensor data and wirelessly transmits it	RP2040 dual-core ARM Cortex M0 + processor; 2.4 GHz wireless LAN; Bluetooth 5.2; 264 KB SRAM; 2 MB flash memory
AC1750 Wireless Dual Band Gigabit Router	Receives data from Raspberry Pi Pico and connects to the computer	Wi-Fi; 5 GHz and 2.4 GHz dual-band communication
ACEIRMC XL6019 DC-DC Boost Converter	Regulates voltage supplied to the electronics on board the rocket	3-35 V input; 5-40 V output; 5 A max current

Graincell Rechargeable Ni-MH AA Batteries	Provides power to the instrumentation package onboard the aircraft	1.5 V nominal voltage; 2800 mAh capacity
--	--	--

Table 2 — Instrumentation and wireless data acquisition components that were used to measure, transmit, and process the collected onboard acceleration data for the rocket launch.

III. Experimental Procedure and Analysis

i) Wireless Data Acquisition Setup

The router was connected via ethernet or Wi-Fi to the computer (receiving end), on which the data was being collected. The corresponding IP address (e.g. IPV4) was found and updated in the Arduino code (C++). This is so that UDP data packages were able to be sent from the router through the MSG port (444) to the computer.

Once the Arduino code was appropriately updated, the desired accelerometer transmission (MPU6050-16g or H3LIS331-400g) was selected in the code, and the appropriate serial port (COM5) was used for the code upload process.

Via a USB-micro-USB data cable, the Raspberry Pi Pico W, (microcontroller), which resided inside the instrumentation housing unit, was connected to the computer. Then the instrumentation power was activated (assuming the batteries were installed). The Arduino code was subsequently uploaded directly into the microcontroller and was then disconnected from the computer after the uploading process was complete. It is critical that the onboard power supply remains active after the microcontroller is disconnected from the computer.

The electronics unit (power converter + accelerometers + microcontroller) was then carefully seated inside the instrumentation housing such that the Dura Lock layers were fully interlocked. The housing cap was likewise inserted on top of the electronics unit. At this point, raw accelerometer data (for each axis) was being sent to the receiving end (with the appropriate IP address), but it was not necessarily being recorded. The instrumentation was then ready for calibration.

ii) Calibration Protocol

The calibration process involved (implicitly) determining the relative orientation between the accelerometer axes and the coordinate frame associated with the instrumentation housing. In conjunction, it accounted for the accelerometer's bias, and (implicitly) the gains associated with each acceleration component.

Three coordinate axes (coordinate frame) were established with respect to the instrumentation housing. The z-axis was oriented to be normal to the housing cap's exposed (top) surface. The x and y axis were established to be (ideally) mutually orthogonal to each other, and the z-axis. In particular, the x and y axis were made to be (roughly) aligned with the respective axis on board the accelerometer of choosing. Note though that the choice of the x and y axis was technically arbitrary, so long as the z-axis was normal to the x-y plane (i.e. the x and y axis need not even be mutually orthogonal, but it is preferable).

Once the relative coordinates were established, calibration could commence. To begin, the x-axis was faced upright (negatively aligned with the gravitational field). The calibration Python script on the receiving end of the data stream was then run. The raw (biased) accelerations reported by the

accelerometer (in m/s^2) were displayed in the Python console. Next, the housing unit was rotated 180° in the y-z plane such that the negative x-axis was faced upright. The Python script was run again, and the raw accelerations were displayed in the console. This process was repeated two more times for the y and z axis respectively, for a total of six calibration measurements (two for each axis).

Once all the accelerations were recorded (in the console), they were manually copy-pasted into the respective input location in the MATLAB code used for post-processing. The post-processing (MATLAB) script was where the calibration algorithm was implemented (see Appendix for details).

iii) Launching & Data Acquisition

With the accelerometer calibration specifications tabulated, the system was nearly ready for testing. The instrumentation housing unit was fitted to the top of the water bottle rocket (opposite to the nozzle).

Next, the rocket was placed inside the test stand and was locked in place via the locking mechanism. After that, the rocket was placed inside the launch tube with the stand keeping it in place on the floor beneath the tube. Then, the bike pump was securely attached to the stand, and so the rocket was ready to be pressurized.

Before the test was ready to commence, the bike pump was pumped such that the internal pressure reading inside the rocket (via the external pressure gauge) reported 30 psi (by default). At the test stand, an experimentalist (A) placed one hand on the stand itself to fix it in place, and another on the rocket/pressure release lever.

Once the rocket was ready to be launched, another experimentalist (B) ran the data recording software (Python script) on the receiving end of the data stream, which writes the raw acceleration data to an excel file. When appropriate, the experimentalist (B) instructed the other (A) to pull on the lever to release the pressurized rocket. After the rocket launched and fell back down, the recording software was terminated.

iv) Data Importing – Post Processing

At this point, an excel file (named: accelerometer data – change file path if needed in the code) was populated with all the raw data including the acceleration components (in m/s^2) and the time stamps (in μs), none of which were yet calibrated. Finally, the excel file was imported into MATLAB (post-processing script) for future analysis.

IV. Results and Discussion

Figure 10 shows the raw accelerometer data in all three axes for the instrumentation package through a full launch cycle at a measurement range of 2-g. There are noticeable “bumps” in the data for all three axes, at approximately the same time stamps, which indicates some unwanted interaction between the rocket and the launch tube.

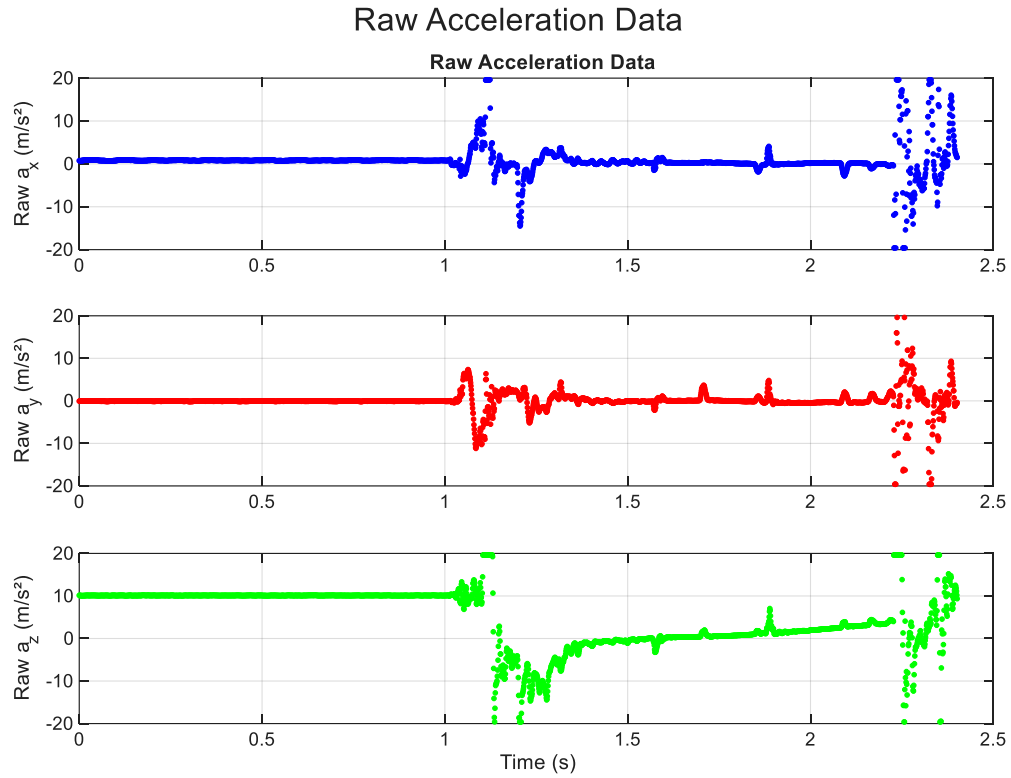


Figure 10 — Raw accelerometer data captured by the low-g sensor during a full launch test with a measurement range of 2-g. The dataset highlights physical interactions with the guide tube (bumps) in all three axes

Additionally, the data at launch and landing reaches the limits of our measurement range, which is seen as clipping, a data point with a rising edge behind it, followed by a data point with a close max value and a falling edge. The effects of gravity are seen with an acceleration of approximately but not exactly 1g in the z axis. This warranted the necessity for the calibration of the axes and expansion of measurement range. Figure 11 shows a calibrated 16-g launch.

Calibrated Acceleration Data

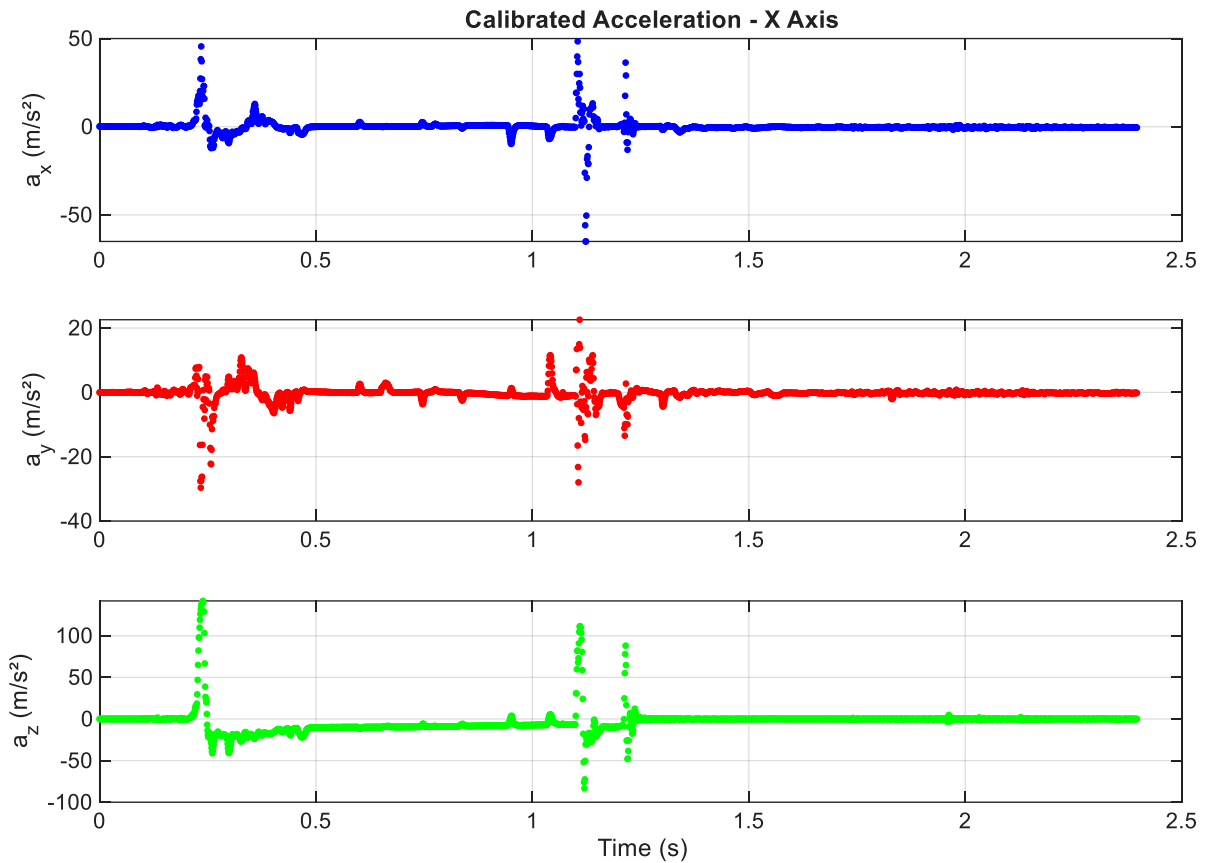


Figure 11 — Calibrated accelerometer data captured by the low-g sensor during a full launch test with a measurement range of 16-g.

We can see the calibration was successful since the acceleration at rest was consistently reading ~ 0 m/s² for all axes which is expected since the error gains of the data were removed, and 1-g is subtracted from the raw received data, leading to a return to baseline. Because of this normalization, any rocket movement will be seen as non-zero acceleration. Even though the measurement range was raised to 16-g, the values recorded at launch and landing still hit the limits of this range, which indicates that another range increase could be beneficial.

Figure 12 shows the data collected from the high g sensor across all three axes at 100g resolution. A change in instrumentation was required to obtain the intended larger measurement range but presented its own nuance.

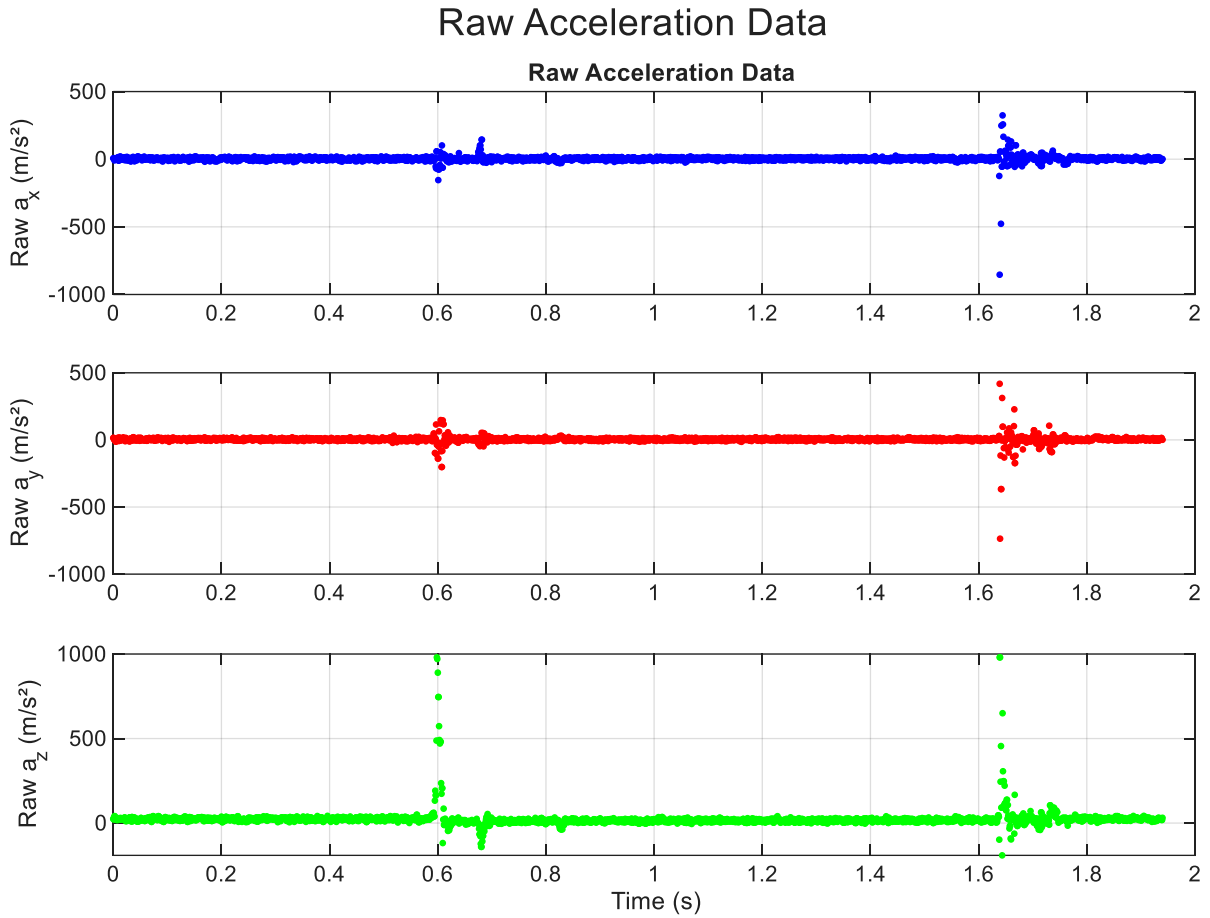


Figure 12- Raw accelerometer data captured by the low-g sensor during a full launch test with a measurement range of 100-g, clearly showing the increased noise level at the baseline while the rocket is stationary

As advertised by the manufacturer, the high g accelerometer exhibits a large baseline noise when stationary or operating at low g's, then the low axis accelerometer. Furthermore, despite the increase in measurement range, there was still clipping observed at launch and landing indicating more measurement resolution was required. *Figure 12* shows data collected in the 400 g resolution:

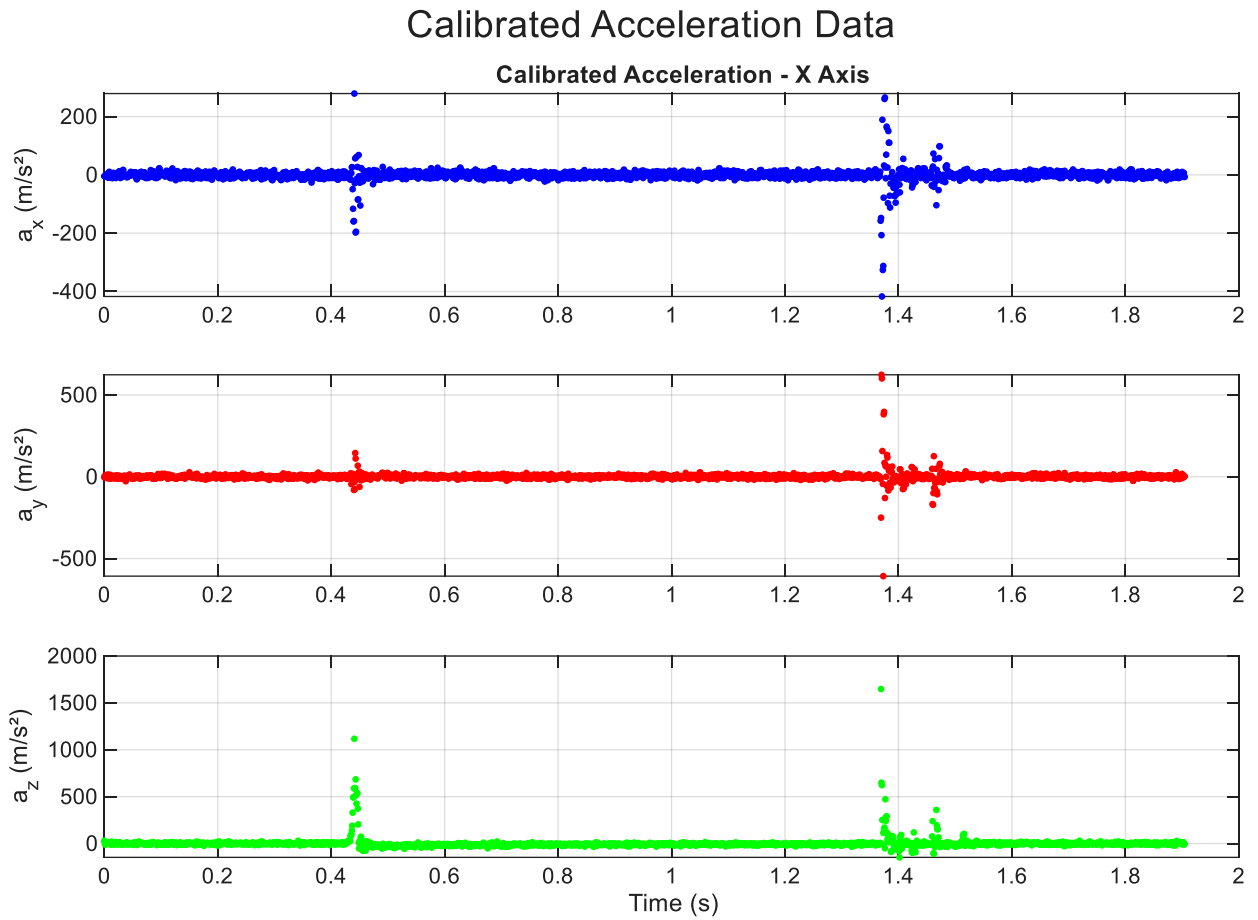


Figure 13 — Calibrated accelerometer data captured by the low-g sensor during a full launch test with a measurement range of 400-g. Shows improved clipping and still demonstrates baseline noise

In the figure, the 400g measurement range captured the full launch boost of the rocket but was too noisy to preserve small scale rocket dynamics in the coast phase, indicating that both accelerometers are crucial for rocket analysis. *Figure 14* shows the coast phase of the launch measured with 16 g resolution

Calibrated Acceleration with Error Bars [0.270 – 1.100] s

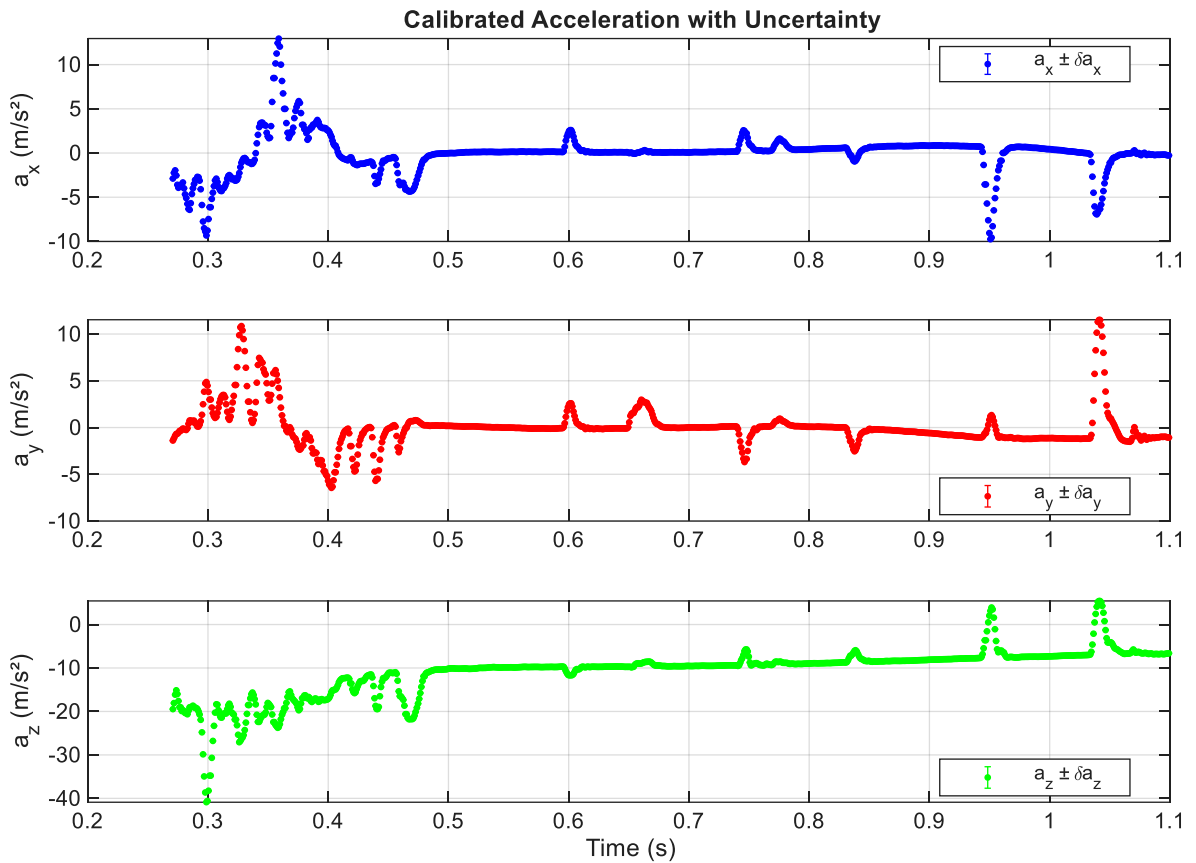


Figure 14 — The coast phase from about 0.27 to 1.1 seconds when captures with the 16-g measurement range

When taking a closer look at the 16-g rocket data, certain parts of the flight trajectory can be noticed. The following figure shows the portion of the flight right after the initial boost and right before landing, known as the coast. At the beginning, a considerable amount of oscillation can be noticed across all three axes. The following period (~ 0.275 - 0.475 s) corresponds to the period where the pressure seal on the bottle rocket has been broken, but the mechanism hasn't fully moved out of the way. Initially, the effects of this period were neglected in the theoretical model but are seen to be non-trivial. The following period (~ 0.475 - 1 s) shows a somewhat linear decrease in Z acceleration, which can be attributed to the lower of the pneumatic breaking effect due to a decrease in speed. The following trend continues until the rocket crashes indicated by the large acceleration spike seen prior.

Figure 15 shows the theoretically predicted accelerations of the rocket across all stages developed by Professor Sidebotham.

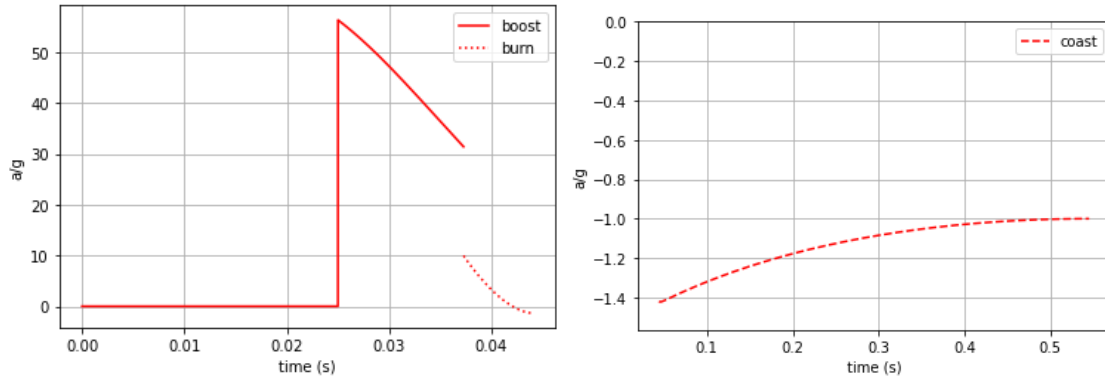


Figure 15 — Theoretical accelerations across the boost and burn stage of the flight modeled by using ideal one-dimension kinematics as developed by Professor Sidebotham

A certain level of alignment between theory and experimental results can be noted. The unit difference should be noted, as experimental data reports acceleration in m/s^2 , while the theoretical model reports acceleration normalized by gravity, i.e. g . The initial acceleration spike due to the boost and burn phase is preserved, meanwhile a pressure/acceleration drop due to the delayed release of the rocket is also noted. Additionally, a decrease in acceleration magnitude was also noticed during the coast phase but is seen to be nonlinear and reaching free fall acceleration.

Figure 16 shows another test trail for data collected under a 30 PSI launch

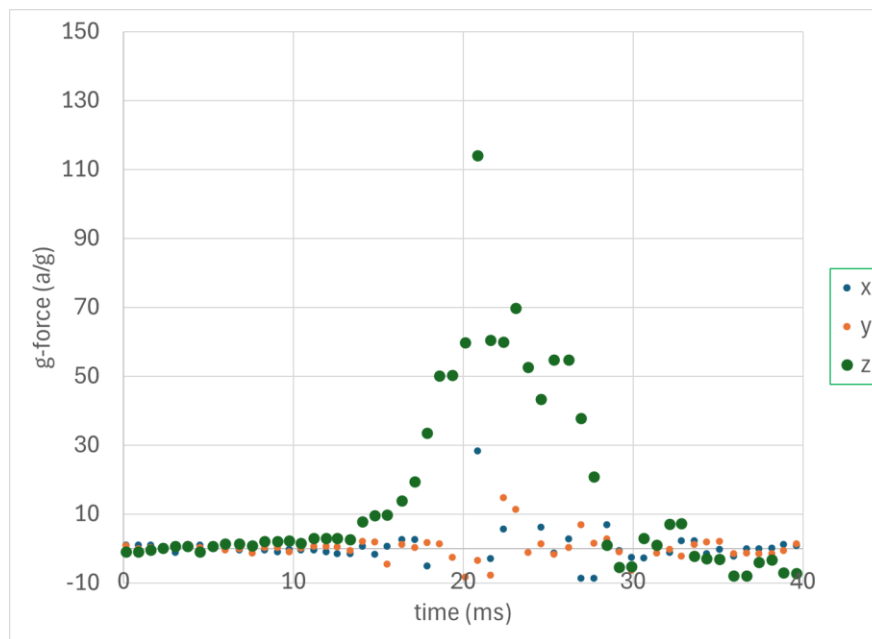


Figure 16 — Calibrated acceleration tracking across all three axes for an initial 30-psi launch pressure. The experimental data still maintains close alignment with the developed theory with the exception of an outlier at the 110-g spike

Figure 17 was highlighted by the project client, as having aligned with their theoretical predictions for the boost phase. The only deviation present is the “outlier” value of ~110-g’s, which could be attributed to an uneven release of pressure from the bottle due to the interference issues presented earlier. Such outlier values were not typically present in other data sets, so analysis of future data will be conducted. Even though the value could be deemed an outlier based on statistical testing, an additional factor of slight changes in the physical setup cannot be ruled out, so the value will be considered for this application.

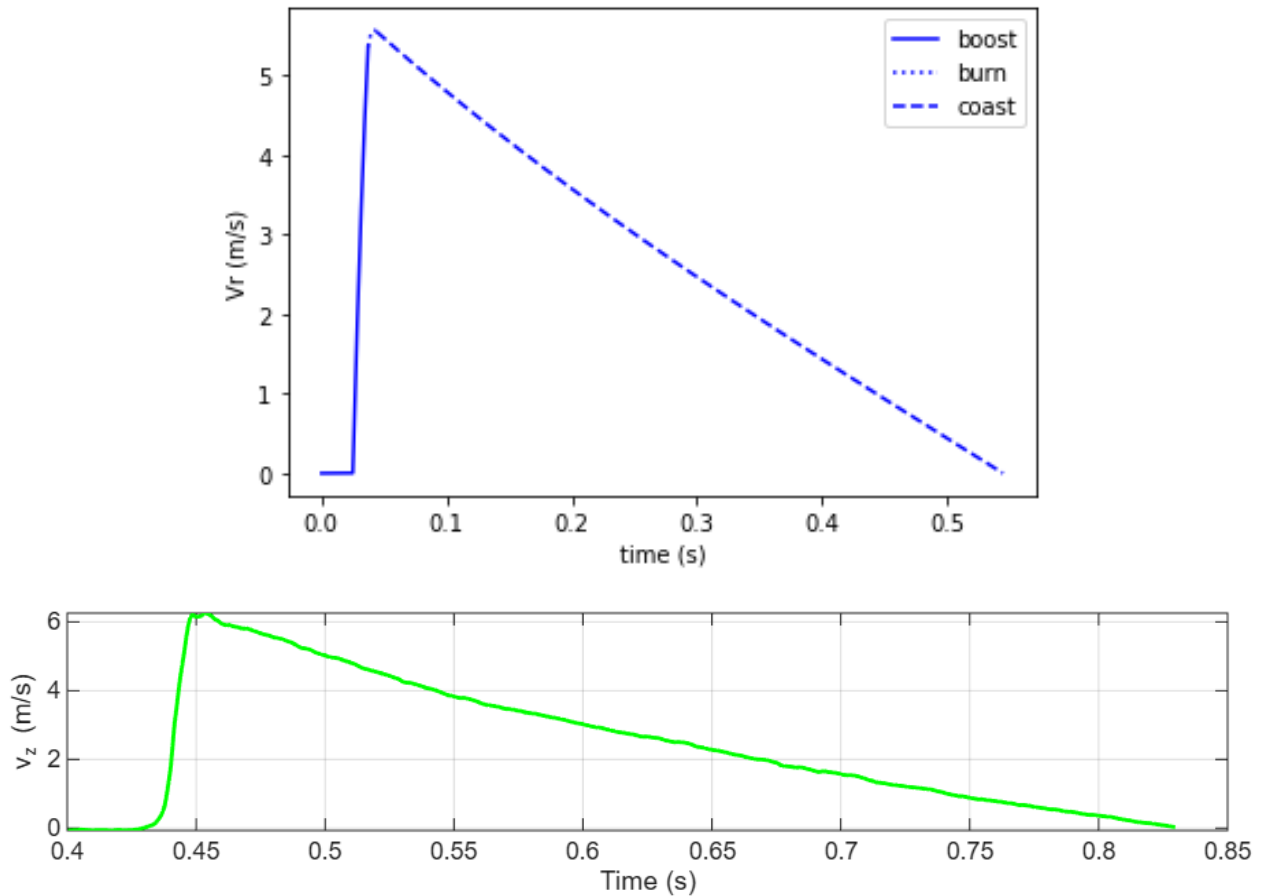


Figure 17 — Comparative analysis of the theoretical modeled trajectory (top) and the experimentally determined velocity (bottom)

In these figures, a general alignment between theory (above) and experimental (below) is still maintained, with a few exceptions. Experimental velocity data was found by numerically integrating 400g calibrated accelerometer data. A similar rise and falling times can be seen in both cases, with a similar peak value. In both cases, the data was cut off for negative velocities. The important takeaway from this testing was the validity of the use of numerical integration on accelerometer data to obtain useful results.

Figure 18 shows position data for the same launch.

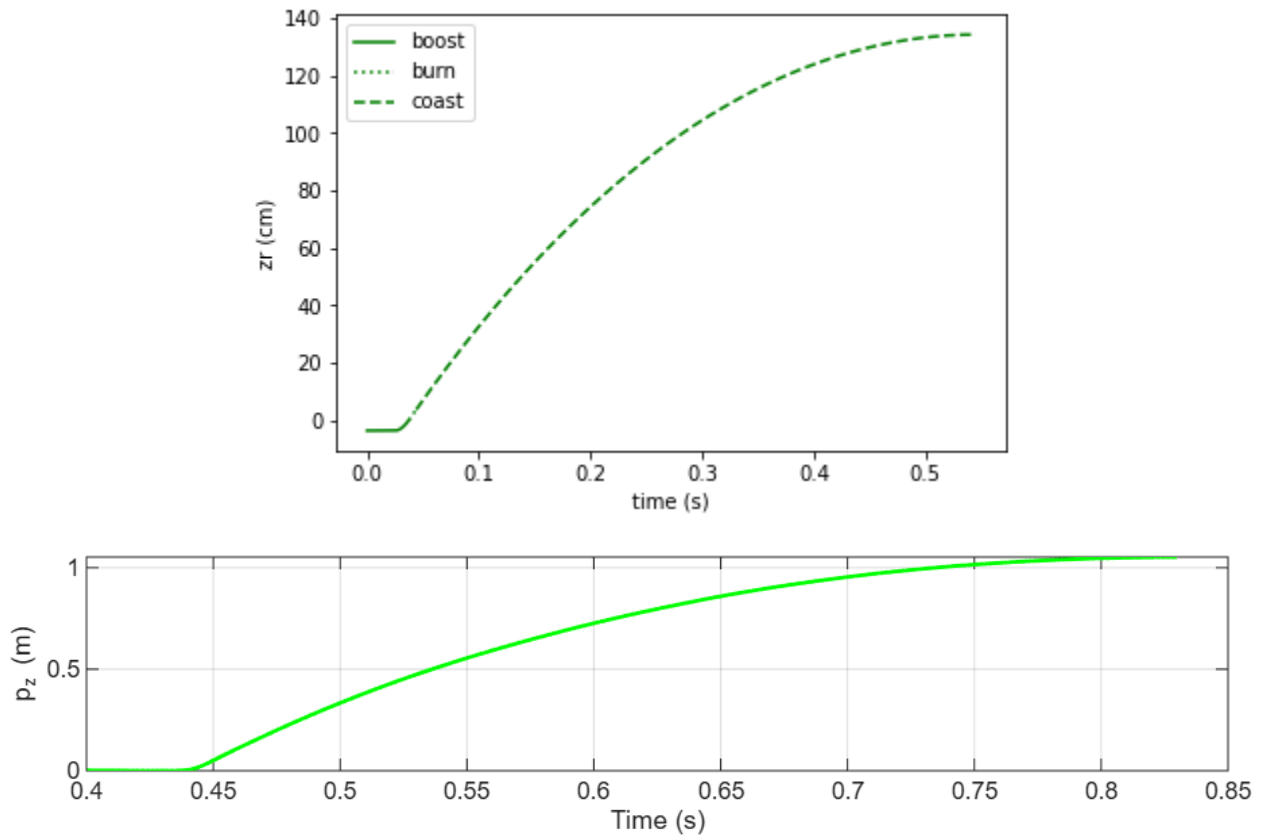


Figure 18 — Trajectory comparing the theoretical flight position (top) with the experimentally determined position (bottom)

Like before, a close similarity is seen between the two data sets (note the difference in units, cm vs m). A similar rise time was noted for both, with the max height found correlating to apogee height. This data set is another validation of two rounds of numerical integration for the high g accelerometer data.

Figure 19 shows data from both accelerometers at rest, with the corresponding uncertainty values.

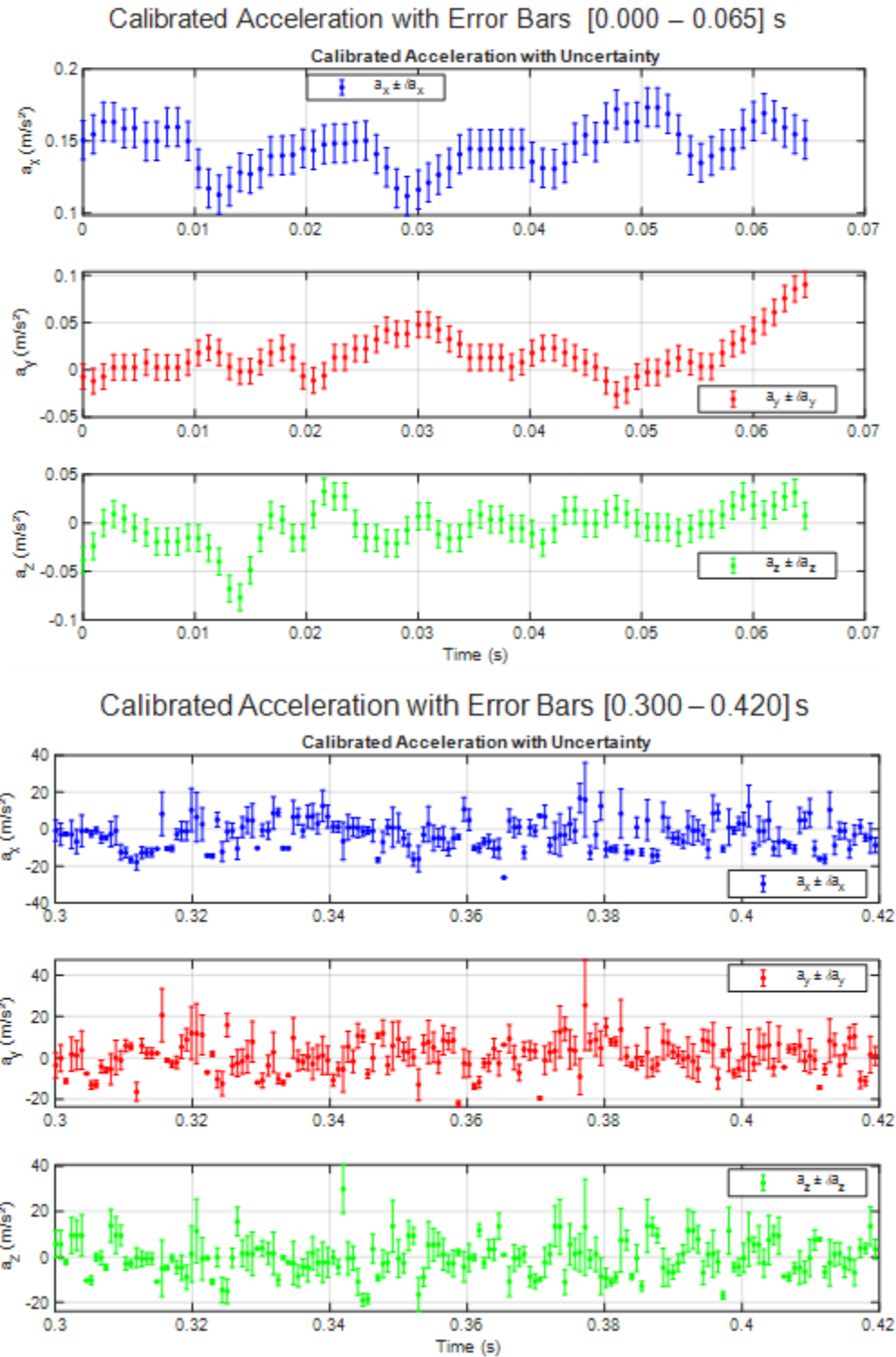


Figure 19 — Comparison of baseline sensor noise when the rocket is stationary. The 16-bit accelerometer is shown at the top and the 12-bit accelerometer is shown at the bottom

In these plots, even at rest, both accelerometers have a spread about 0 m/s², but the high g has a spread on several orders of magnitude greater. This can partly be attributed to the high g accelerometer having an increased measurement range, but smaller precision due to less measurement bits. The

following noise is typical in accelerometers, and results in the inability for collected data to be used for numerical integration. As shown earlier testing with high g accelerometers yielded reasonable integration output, so an analysis on both should be conducted. *Figure 20* and *Figure 21* show the velocity and position data for the 400g, and 16g accelerometers respectively.

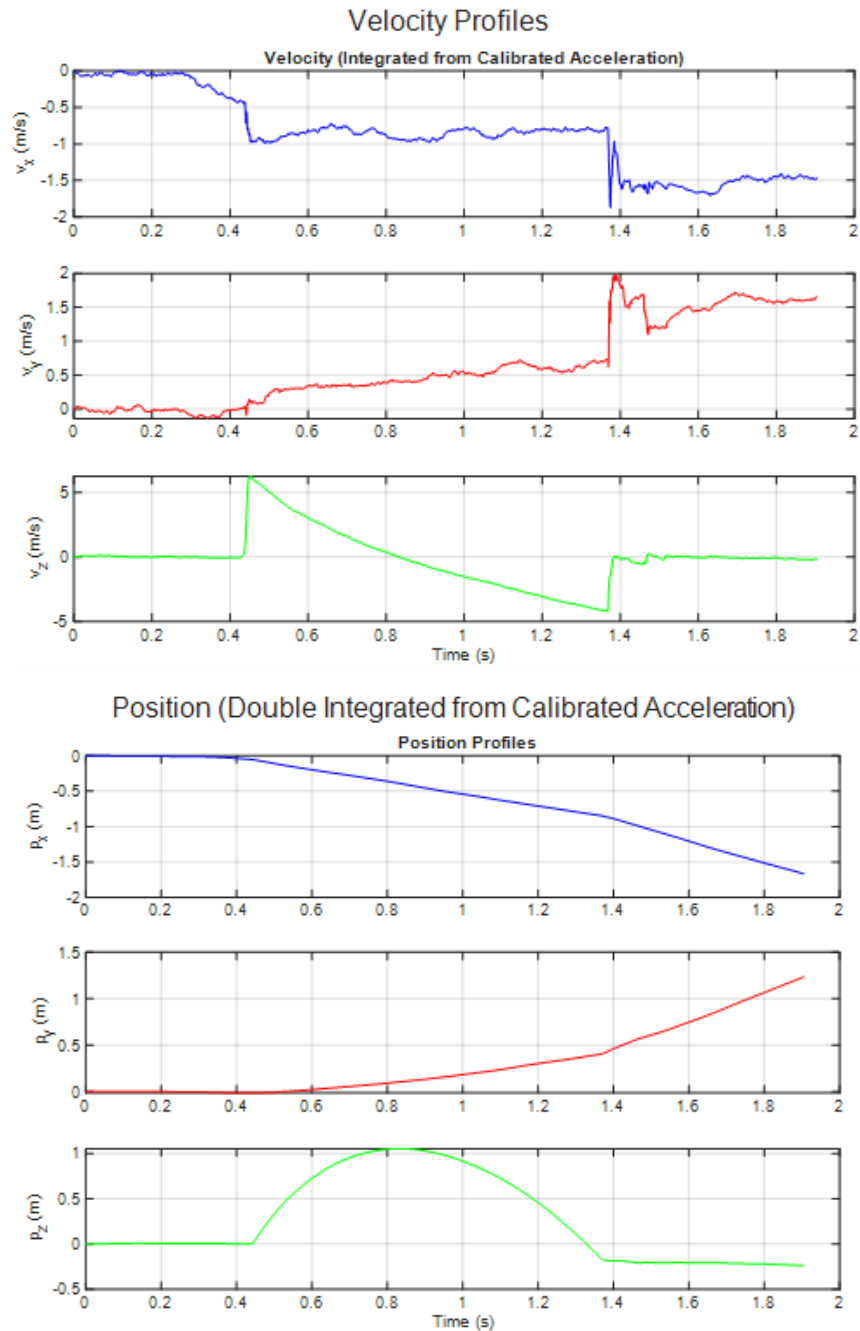


Figure 20 — Complete trajectory showing the integrated velocity (top) and the double integrated position (bottom) from the calibrated 400-g accelerometer data.

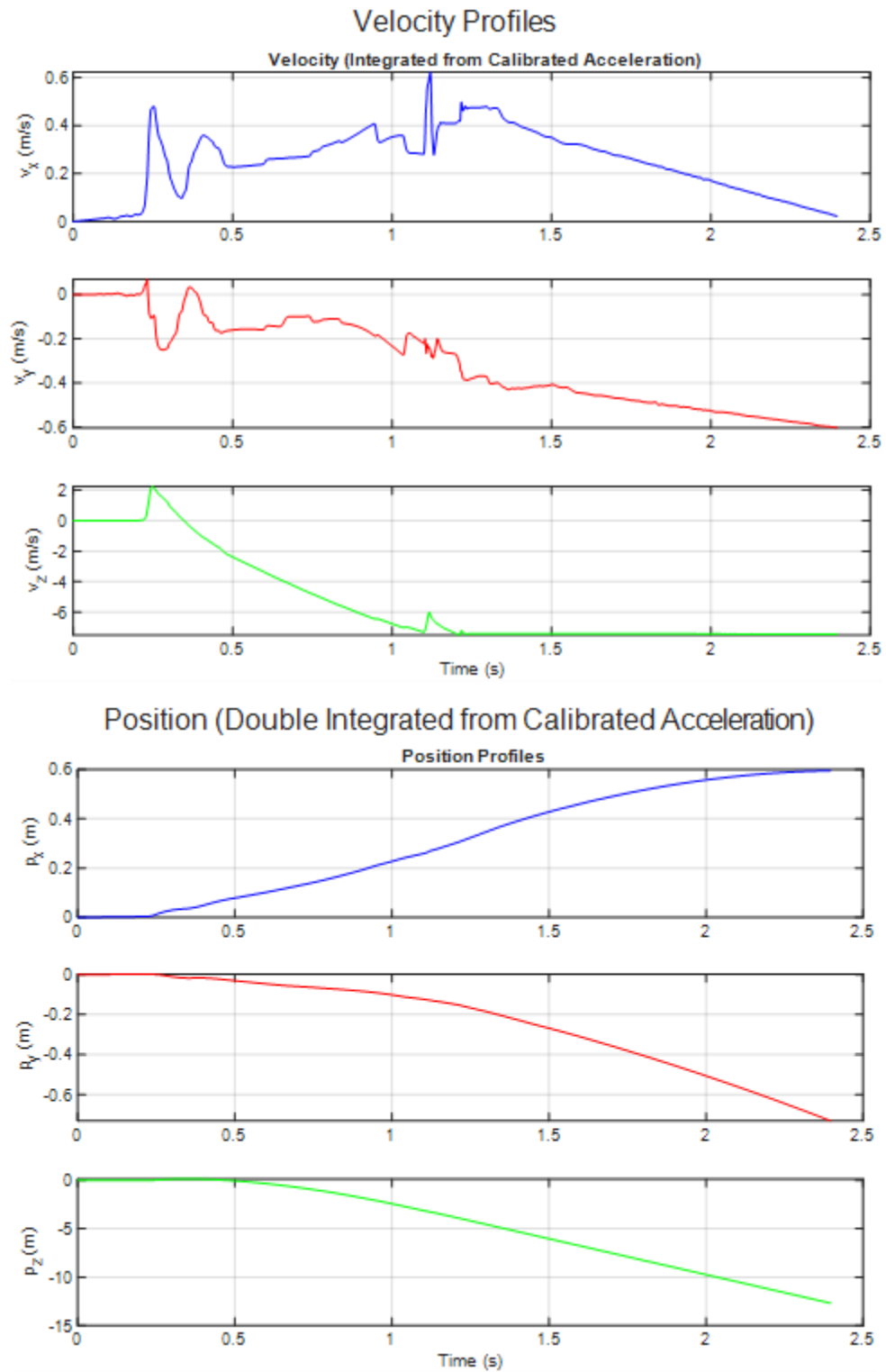


Figure 21 — Complete trajectory showing the integrated velocity (top) and the double integrated position (bottom) from the calibrated 400-g accelerometer data.

In these graphs it can be clearly seen that the high g data can produce accurate position and velocity trajectories for the z direction, while the low g data is incapable of accurate prediction in any of its axes. High g data showed a clean apogee for the launch, and most importantly a return to a baseline height of close to zero (deviation of 0.25 m is 1-2 orders of magnitude smaller than a 12m deviation). Despite this relative success, it should still be noted that the x and y displacements across all tests did not return to zero, indicating some limitations in this estimation method. The difference in the noise of each device, coupled with the clipping seen earlier, could be the reason for the success of one instrument over the other.

Analyses of results across different accelerometers and axes, indicated the validity of use of each device in certain applications. High G data was able to accurately capture the relatively massive boost phase of the rocket, and the overall dynamics. Low G data was able to detect instances of rocket collisions with the launch tube, and the different rocket dynamics at each phase of the launch. Eventual time syncing of these data sets could yield a data set with reductions to error across all aspects.

V. Conclusion

The experimental air rocket test stand successfully achieved its primary objective of finding a simple method of measuring rocket position and acceleration within a controlled environment. The apparatus with the 2.3-m vertical guide tube effectively constrained the rocket's motion to a single direction. The crucial component of this design was the integration of a pneumatic braking system with a 5-cm vent hole at the top. This along with the fixed 9-cm diameter of the nose cone restricted airflow in front of the rocket to reduce its peak velocity, preventing the rocket from colliding with the top of the apparatus, which prevents damaging the onboard instrumentation, as well as helping capture the entirety of the rocket's motion at higher pressures.

The numerical analysis was heavily dependent on the dual-accelerometer configuration with the low-g Adafruit MPU6050 and the high-g Adafruit H3LIS331. Although the 16-g range on the low sensor was very precise for finding any rocket interactions with the rest of the apparatus, it was not suitable for gathering data for the full trajectory as it experienced severe clipping during both the initial launch and final landing. On the other hand, the 400-g range of the high-g Adafruit allowed for the complete capturing of the launch/boost phase with no clipping and demonstrated a good alignment with developed theoretical flight models. There was small variance that was observed during a 30-psi launch, where a 110-g outlier value was observed. This deviation was attributed to some kind of interference in the physical system or an uneven release of pressure.

The study also validated the implementation of integration on calibrated accelerometer data to find meaningful velocity and position values. Double integration of the 400-g sensor generated a launch image that included a clear apogee height, and showed the rocket's return to a height of approximately 0-m. In comparison, the low-g dataset did not provide very accurate position data that could be used to provide significant information about the system's behavior.

Though the high-g accelerometer is very successful, an important limitation to consider is its baseline noise when the system is at rest. Because the high-g accelerometer has fewer bits, there is significant noise when it experiences low-g's. This makes the data acquired while the system is at rest, and the data that is acquired in the coasting phase too noisy to use reliably. Contrasting this, is the low-g accelerometer which thrives where the high-g accelerometer struggles but struggles where the high-g accelerometer thrives. For example, the low-g accelerometer picked up on a non-trivial oscillation between 0.275 and 0.475 seconds that was found to correspond with the exact moment where the pressure seal on the bottle broke.

Ultimately, the instrumented test stand effectively helped improve the education value and close the gap between theoretical concepts and hands-on experimentation.

VI. Future Work

There are several future improvements to be made to the apparatus as well as the instrumentation of the system. The primary instrumentation/software priority is to establish protocol to sync times between the independent datasets of the accelerometers. Developing a method to align the data will allow future researchers, educators, and students to combine the benefits of the 400-g and the 16-g sensor when

appropriate, using the 16-g sensor when tracking the launch and crash, and the 16-g sensor for the coasting phase. Another crucial addition to the instrumentation would be an external range finder situated above the pneumatic braking vent that actively tracks the position of the rocket as it is launched. This provides a baseline to verify both the double integration method used to find position from accelerometer data, as well as Professor Sidebotham's theoretical flight models.

Mechanically, the primary additions should focus on the user's ease of use and the improvement or expansion of physical limits. The current layout of the instrumentation package should be redesigned to allow for easier assembly and disassembly of the electronics as well as easy adjustment of diameter for the pneumatic braking system. Modifying the cap in this way will allow for easier access to the Raspberry Pi Pico, the boost converter, and the battery pack in between trials.

Additionally, the launch platform experiences a strong impulsive force at the rocket's release. Future work on the launch mechanism itself should focus on the stabilization during this period to guarantee uniform launch conditions for every trial. The apparatus can also be run as a water-rocket, but that would require extensive waterproofing of the entire system as well as protective shielding for the user to not experience backslash. Once these are in place, the current work that is focused on an air-rocket case can easily be implemented to collect data for an experimental water rocket.

APPENDIX

Calibration and Error Analysis

In order to determine the true acceleration experienced by the accelerometer relative to the co-moving coordinate system attached to the instrumentation housing unit, the accelerometer readings need to be calibrated against a reference.

We assume not only that the accelerometer coordinates are rotated relative to co-moving coordinate system attached to the instrumentation unit, but that there is an inherent relative scaling in conjunction with a constant bias (offset) between them as well. In particular, provided that $\mathbf{a} = [a_1, a_2, a_3]^T$ is the desired (calibrated) acceleration vector, $\bar{\mathbf{a}} = [\bar{a}_1, \bar{a}_2, \bar{a}_3]^T$ is the measured (raw) acceleration from the accelerometer, $\mathbf{b} = [b_1, b_2, b_3]^T$ is the (constant) bias vector, and $\mathbf{C} = [c_{ij}]_{1 \leq i, j \leq 3}$ is the calibration matrix, this assumption translates to the matrix-vector equation given by $\mathbf{a} = \mathbf{C}(\bar{\mathbf{a}} - \mathbf{b})$. In coordinates, this is equivalent to:

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix} \begin{bmatrix} \bar{a}_1 - b_1 \\ \bar{a}_2 - b_2 \\ \bar{a}_3 - b_3 \end{bmatrix}$$

In order to determine the bias vector and the calibration matrix, we will take advantage of the fact that the accelerometer attempts to report the (negative of the) specific force it experiences due to gravity. Specifically, consider the coordinate frame of the accelerometer $\{\bar{\mathbf{e}}_1, \bar{\mathbf{e}}_2, \bar{\mathbf{e}}_3\}$ and that of the instrumentation housing $\{\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3\}$. Let the gravitational field vector be aligned with $\mathbf{e}_i$ (for some orientation $i = 1, 2, 3$). When the instrumentation is in this orientation and at rest, the accelerometer will report an acceleration $\bar{\mathbf{a}}_i$ with component coefficients $\bar{a}_{ij}$. To be clear, $\bar{a}_{ij}$ denotes the j^{th} component coefficient of the i^{th} raw acceleration rest measurement given by $\bar{\mathbf{a}}_i$. This can all be visualized in the illustration shown below:

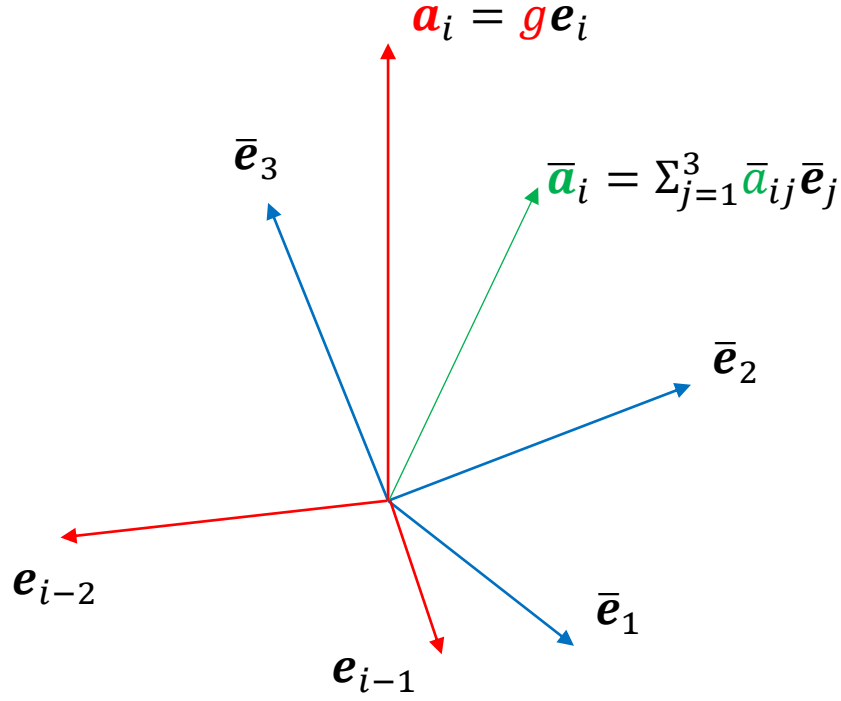


Figure _: Relative Accelerometer Coordinate Orientation

To start, the bias vector can be determined using a neat trick. Suppose in the i^{th} orientation (as shown above), the $\mathbf{e}_i$ basis vector is negatively aligned with the gravitational field (i.e. in the upright position). Crucially, we note that the measured acceleration can be broken up into an unbiased/true component plus the bias itself. That is to say, $\bar{\mathbf{a}}_i = \bar{\mathbf{a}}_i^{\text{true}} + \mathbf{b}$. To be even more precise, in this positive orientation, $\bar{\mathbf{a}}_{\uparrow i} = \bar{\mathbf{a}}_{\uparrow i}^{\text{true}} + \mathbf{b}$. Likewise, in the negative orientation (i.e. $\mathbf{e}_i$ is positively aligned with the gravitational field), the measured acceleration can be given by $\bar{\mathbf{a}}_{\downarrow i} = \bar{\mathbf{a}}_{\downarrow i}^{\text{true}} + \mathbf{b}$, such that the vector sum $\bar{\mathbf{a}}_{\uparrow i}^{\text{true}} + \bar{\mathbf{a}}_{\downarrow i}^{\text{true}} = \mathbf{0}$ (in theory). If not clear, this is because the true accelerations are direct (possibly scaled) measurements of the local gravitational field, and so because those accelerations are oriented oppositely relative to each other, their sum must return null. Since the bias vector $\mathbf{b}$ is assumed to be constant, it can be solved for as such:

$$\mathbf{b} = \frac{\bar{\mathbf{a}}_{\uparrow i} + \bar{\mathbf{a}}_{\downarrow i}}{2}$$

Although this equation should work in theory, in practice, due to inherent sensor noise, averaging the bias vector measurement across all 3 possible orientation pairs, a potentially more accurate measurement of the bias vector, should be performed as follows:

$$\mathbf{b} = \frac{1}{6} \sum_{i=1}^3 (\bar{\mathbf{a}}_{\uparrow i} + \bar{\mathbf{a}}_{\downarrow i})$$

Equivalently, in coordinates:

$$b_j = \frac{1}{6} \sum_{i=1}^3 (\bar{a}_{\uparrow ij} + \bar{a}_{\downarrow ij})$$

With the bias vector out of the way, it's time to tackle the calibration matrix $\mathbf{C}$. Consider again the i^{th} orientation as analyzed before. In this orientation, it is desired (for now) that the calibrated accelerometer report $\mathbf{a} = [g\delta_{i1}, g\delta_{i2}, g\delta_{i3}]^T$, where δ_{ij} is the Kronecker Delta. Under the assumption that $\mathbf{C}$ is an invertible matrix, it directly follows that:

$$\begin{bmatrix} g\delta_{i1} \\ g\delta_{i2} \\ g\delta_{i3} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix} \begin{bmatrix} \bar{a}_{i1} - b_1 \\ \bar{a}_{i2} - b_2 \\ \bar{a}_{i3} - b_3 \end{bmatrix}$$

Or equivalently:

$$\begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}^{-1} \begin{bmatrix} \delta_{i1} \\ \delta_{i2} \\ \delta_{i3} \end{bmatrix} = \frac{1}{g} \begin{bmatrix} \bar{a}_{i1} - b_1 \\ \bar{a}_{i2} - b_2 \\ \bar{a}_{i3} - b_3 \end{bmatrix}$$

An exercise in linear algebra suggests the following relationship:

$$\begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}^{-1} = \frac{1}{g} \begin{bmatrix} \bar{a}_{11} - b_1 & \bar{a}_{21} - b_1 & \bar{a}_{31} - b_1 \\ \bar{a}_{12} - b_2 & \bar{a}_{22} - b_2 & \bar{a}_{32} - b_2 \\ \bar{a}_{13} - b_3 & \bar{a}_{23} - b_3 & \bar{a}_{33} - b_3 \end{bmatrix}$$

Or equivalently:

$$\begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix} = g \begin{bmatrix} \bar{a}_{11} - b_1 & \bar{a}_{21} - b_1 & \bar{a}_{31} - b_1 \\ \bar{a}_{12} - b_2 & \bar{a}_{22} - b_2 & \bar{a}_{32} - b_2 \\ \bar{a}_{13} - b_3 & \bar{a}_{23} - b_3 & \bar{a}_{33} - b_3 \end{bmatrix}^{-1}$$

This is in essence the calibration algorithm in full. However, it is preferable for the calibrated accelerometer to report zero acceleration in every orientation when at rest. Unfortunately, in general, this is quite difficult to achieve due to the possibility that coordinate frame itself gyrates when the instrumentation is in motion. Because the motion of the rocket, and thus the instrumentation housing, is approximately restricted to vertical motion along the launch tube, it will be assumed that the coordinate gyration is negligible. In other words, the orientation is fixed. With this assumption, the (negative of the) gravitational field given by $g\mathbf{e}_3$ can be subtracted off to form the completed calibration algorithm as such:

$$\begin{bmatrix} \mathbf{a}_1 \\ \mathbf{a}_2 \\ \mathbf{a}_3 \end{bmatrix} = \mathbf{g} \begin{bmatrix} \bar{a}_{11} - b_1 & \bar{a}_{21} - b_1 & \bar{a}_{31} - b_1 \\ \bar{a}_{12} - b_2 & \bar{a}_{22} - b_2 & \bar{a}_{32} - b_2 \\ \bar{a}_{13} - b_3 & \bar{a}_{23} - b_3 & \bar{a}_{33} - b_3 \end{bmatrix}^{-1} \begin{bmatrix} \bar{a}_1 - b_1 \\ \bar{a}_2 - b_2 \\ \bar{a}_3 - b_3 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ \mathbf{g} \end{bmatrix}$$

The low-g accelerometer (MPU6050) internally uses a 16-bit signed sensor output within the $\pm 16g$ range. Likewise, the high-g accelerometer (H3LIS331) internally uses a 12-bit signed sensor output within the $\pm 400g$ range. The uncertainties in the raw accelerations for both of these accelerometers can thus be cited as such:

$$\delta \bar{a}_{16g} = \frac{2(16g)}{2^{16}} \approx 4.79 \times 10^{-3} \text{ m/s}^2$$

$$\delta \bar{a}_{400g} = \frac{2(400g)}{2^{12}} \approx 1.91 \times 10^0 \text{ m/s}^2$$

Now, the propagated error in the final accelerometer reading has several components to it, which all stem from the inherent error of the raw data. To see this, recall that the calibrated acceleration is given by $\mathbf{a} = \mathbf{C}(\bar{\mathbf{a}} - \mathbf{b}) - \mathbf{g}\mathbf{e}_3$, and so any *small* deviation in $\mathbf{a}$ can be given by the product rule:

$$\delta \mathbf{a} = (\delta \mathbf{C})(\bar{\mathbf{a}} - \mathbf{b}) + \mathbf{C}(\delta \bar{\mathbf{a}} - \delta \mathbf{b})$$

To be explicit, define:

$$\mathbf{A} := \mathbf{g}\mathbf{C}^{-1} = \begin{bmatrix} \bar{a}_{11} - b_1 & \bar{a}_{21} - b_1 & \bar{a}_{31} - b_1 \\ \bar{a}_{12} - b_2 & \bar{a}_{22} - b_2 & \bar{a}_{32} - b_2 \\ \bar{a}_{13} - b_3 & \bar{a}_{23} - b_3 & \bar{a}_{33} - b_3 \end{bmatrix}$$

It follows that $\mathbf{C} = \mathbf{g}\mathbf{A}^{-1}$, and so $\delta \mathbf{C} = -\mathbf{g}\mathbf{A}^{-1}(\delta \mathbf{A})\mathbf{A}^{-1} = -\mathbf{g}^{-1}\mathbf{C}(\delta \mathbf{A})\mathbf{C}$. Therefore, we can write furthermore that:

$$\delta \mathbf{a} = \mathbf{C}(\delta \bar{\mathbf{a}} - \delta \mathbf{b}) - \mathbf{g}^{-1}\mathbf{C}(\delta \mathbf{A})\mathbf{C}(\bar{\mathbf{a}} - \mathbf{b}) = \mathbf{C} \left((\delta \bar{\mathbf{a}} - \delta \mathbf{b}) - \mathbf{g}^{-1}(\delta \mathbf{A})(\mathbf{a} + \mathbf{g}\mathbf{e}_3) \right)$$

The variation in $\bar{\mathbf{a}}$ (i.e. $\delta \bar{\mathbf{a}}$) is simply the uncertainties in the raw accelerations for either the MPU6050 or the H3LIS331 accelerometer. That is, $\delta \bar{a}_i = \delta \bar{\mathbf{a}} = \pm \delta \bar{a}_{16g}$ or $\pm \delta \bar{a}_{400g}$ depending on the use case. Likewise, inside the $\delta \mathbf{A}$ matrix, $\delta \bar{a}_{ij} = \delta \bar{\mathbf{a}}$ as well.

Where things get slightly more interesting is in determining $\delta \mathbf{b}$. In order to work out the bias uncertainty, we use an upper bound via the Euclidean norm:

$$\delta b_j = \pm \sqrt{\sum_{i=1}^3 \left[\left(\frac{\partial b_j}{\partial \bar{a}_{\uparrow ij}} \delta \bar{a}_{ij} \right)^2 + \left(\frac{\partial b_j}{\partial \bar{a}_{\downarrow ij}} \delta \bar{a}_{ij} \right)^2 \right]} = \pm \sqrt{3 \left[\left(\frac{\delta \bar{a}}{6} \right)^2 + \left(\frac{\delta \bar{a}}{6} \right)^2 \right]} = \pm \frac{\delta \bar{a}}{\sqrt{6}}$$

It therefore follows that:

$$\delta \mathbf{A} = \begin{bmatrix} \delta \bar{a}_{11} - \delta b_1 & \delta \bar{a}_{21} - \delta b_1 & \delta \bar{a}_{31} - \delta b_1 \\ \delta \bar{a}_{12} - \delta b_2 & \delta \bar{a}_{22} - \delta b_2 & \delta \bar{a}_{32} - \delta b_2 \\ \delta \bar{a}_{13} - \delta b_3 & \delta \bar{a}_{23} - \delta b_3 & \delta \bar{a}_{33} - \delta b_3 \end{bmatrix} = \left(1 \pm \frac{1}{\sqrt{6}} \right) \begin{bmatrix} \delta \bar{a} & \delta \bar{a} & \delta \bar{a} \\ \delta \bar{a} & \delta \bar{a} & \delta \bar{a} \\ \delta \bar{a} & \delta \bar{a} & \delta \bar{a} \end{bmatrix}$$

Likewise...

$$\delta \bar{a} - \delta b = \left(1 \pm \frac{1}{\sqrt{6}} \right) \delta \bar{a}$$

If we now synthesize everything together, we can obtain a conservative upper bound for the uncertainty as given below:

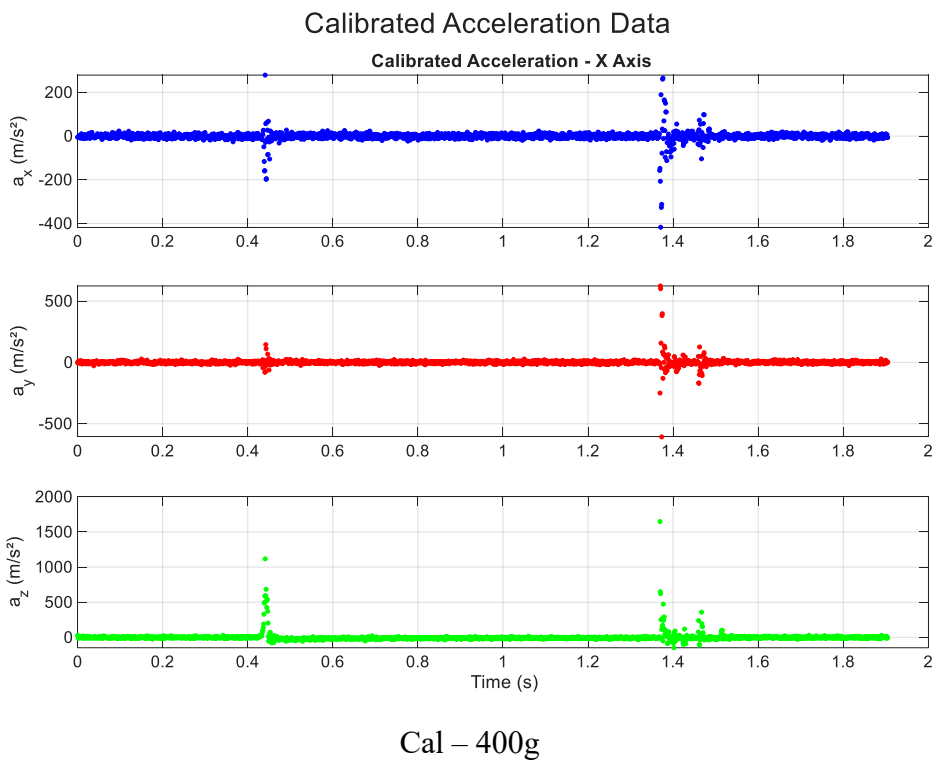
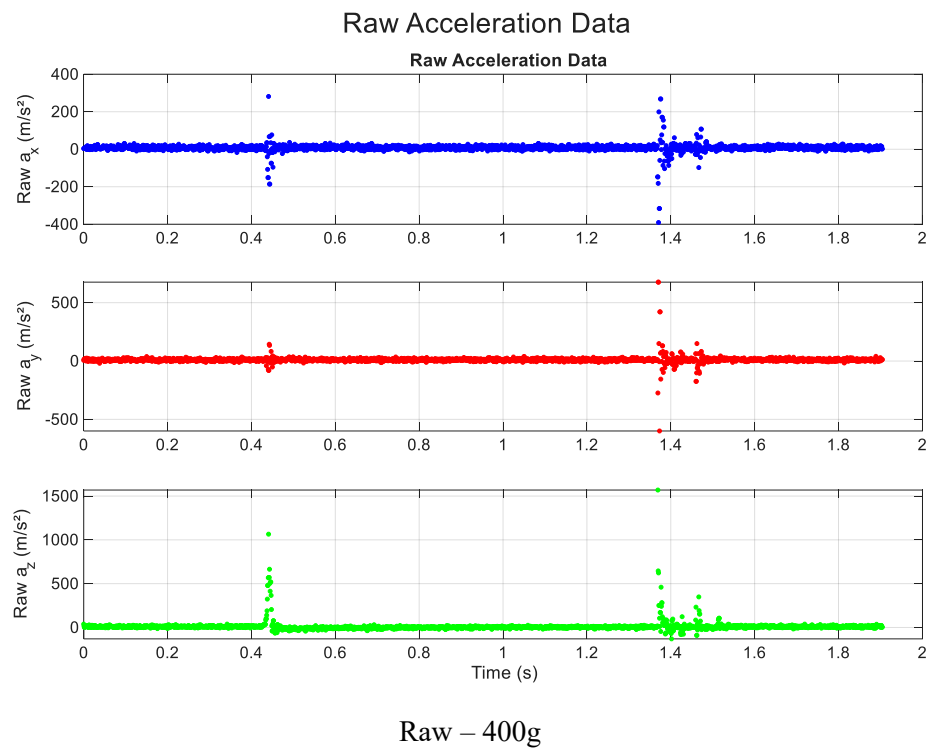
$$\delta \mathbf{a} = \mathbf{C} \left(\left(1 + \frac{1}{\sqrt{6}} \right) \delta \bar{a} + \mathbf{g}^{-1} \left(1 + \frac{1}{\sqrt{6}} \right) \begin{bmatrix} \delta \bar{a} & \delta \bar{a} & \delta \bar{a} \\ \delta \bar{a} & \delta \bar{a} & \delta \bar{a} \\ \delta \bar{a} & \delta \bar{a} & \delta \bar{a} \end{bmatrix} (\mathbf{a} + \mathbf{g} \mathbf{e}_3) \right)$$

More explicitly, after simplification:

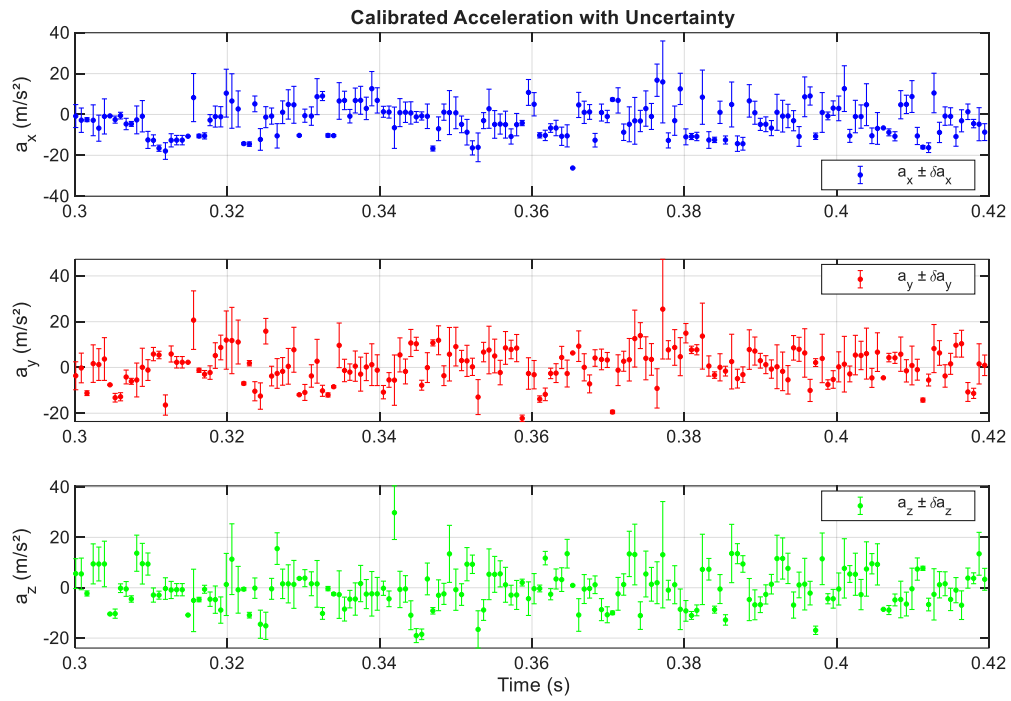
$$\begin{bmatrix} \delta a_1 \\ \delta a_2 \\ \delta a_3 \end{bmatrix} = \left(1 + \frac{1}{\sqrt{6}} \right) \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix} \left(2 \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} + \mathbf{g}^{-1} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} \right) \delta \bar{a}$$

This is the uncertainty formula for the calibrated acceleration(s), and it is what has been used in the error analysis (in MATLAB).

PLOTS

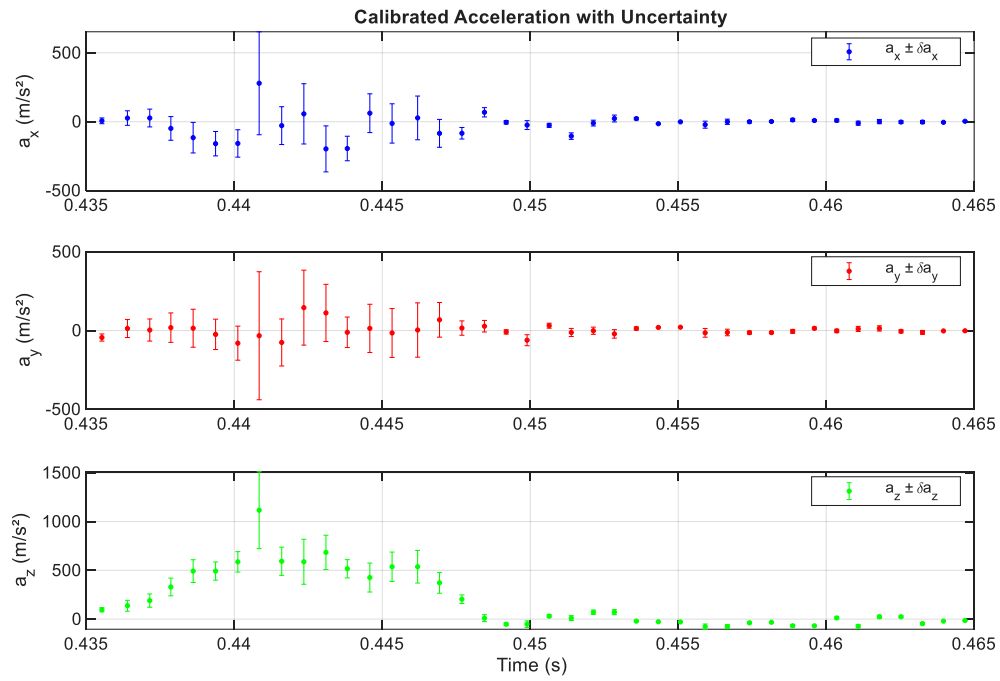


Calibrated Acceleration with Error Bars [0.300 – 0.420] s



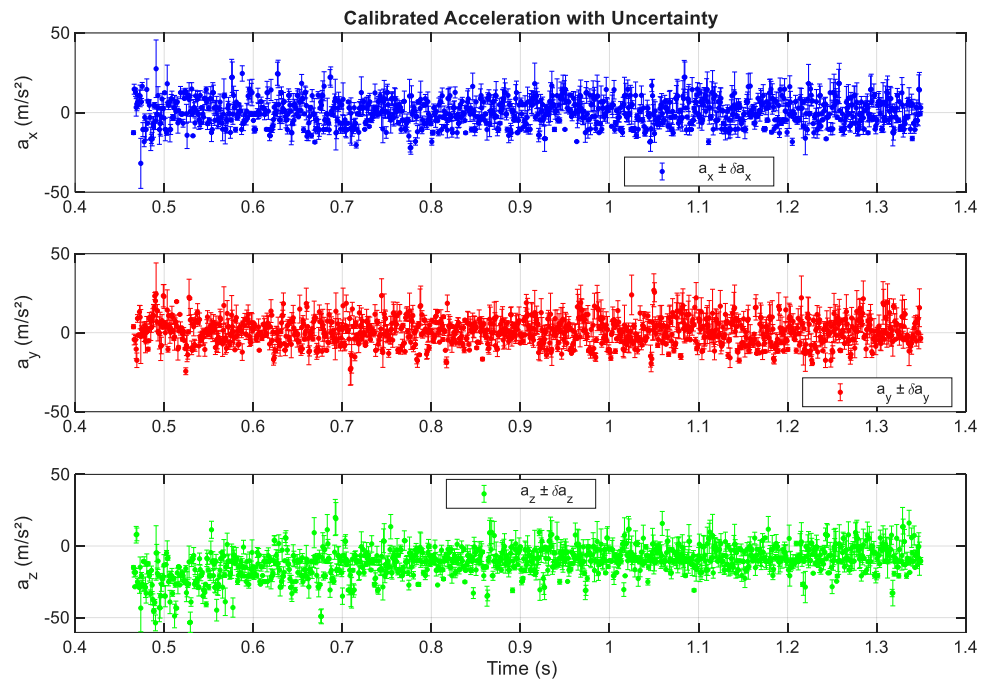
Cal – 400g - rest

Calibrated Acceleration with Error Bars [0.435 – 0.465] s



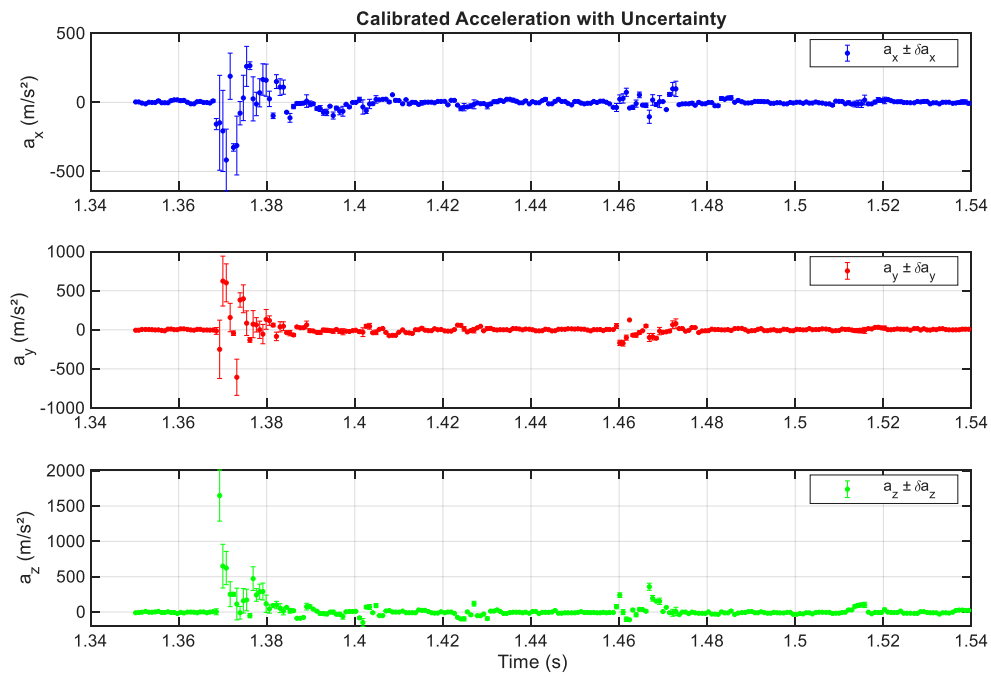
Cal – 400g – Boost & Burn

Calibrated Acceleration with Error Bars [0.465 – 1.350] s



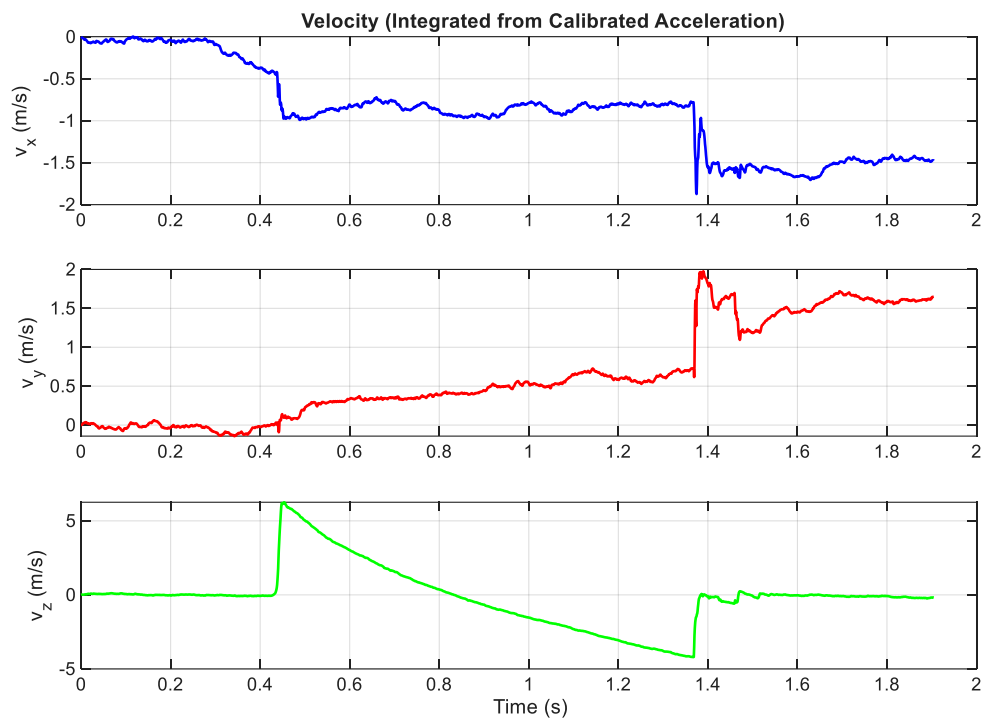
Cal – 400g – Coast

Calibrated Acceleration with Error Bars [1.350 – 1.540] s



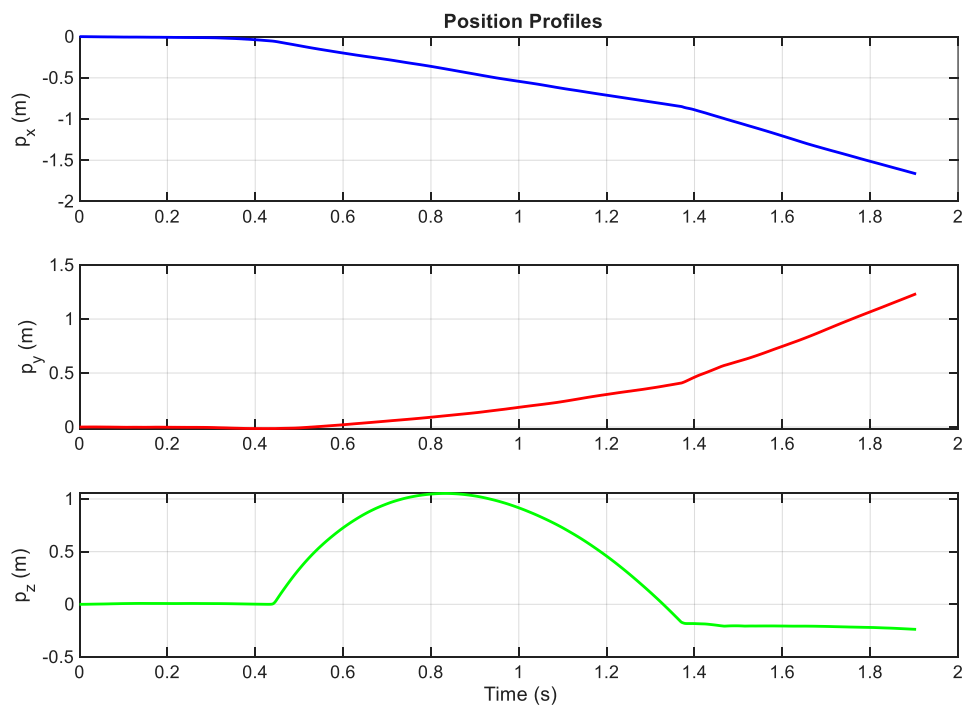
Cal - 400g – Crash

Velocity Profiles



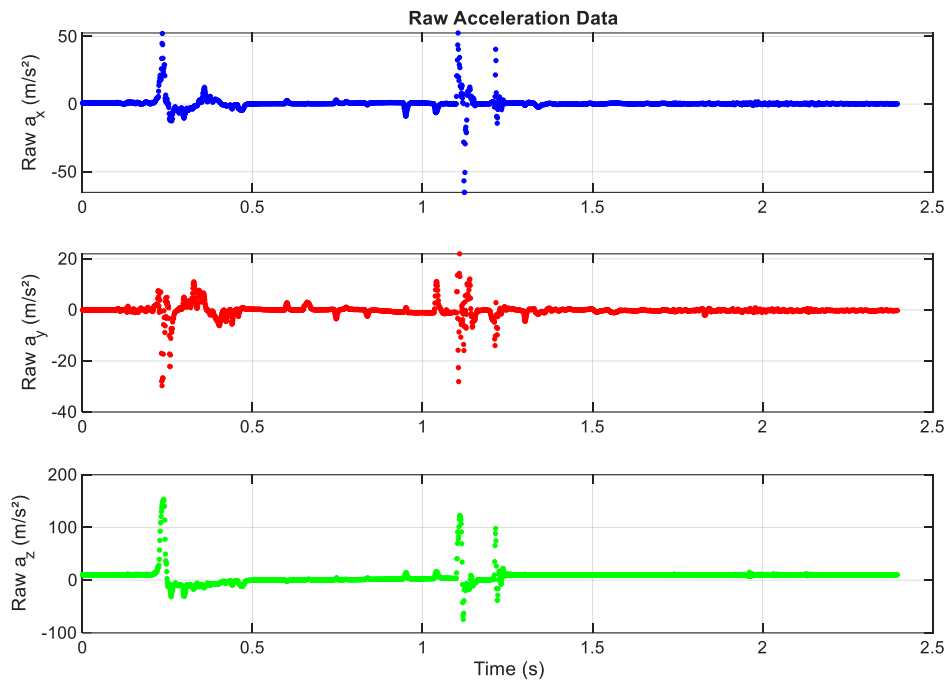
Cal – Velocity – 400g

Position (Double Integrated from Calibrated Acceleration)



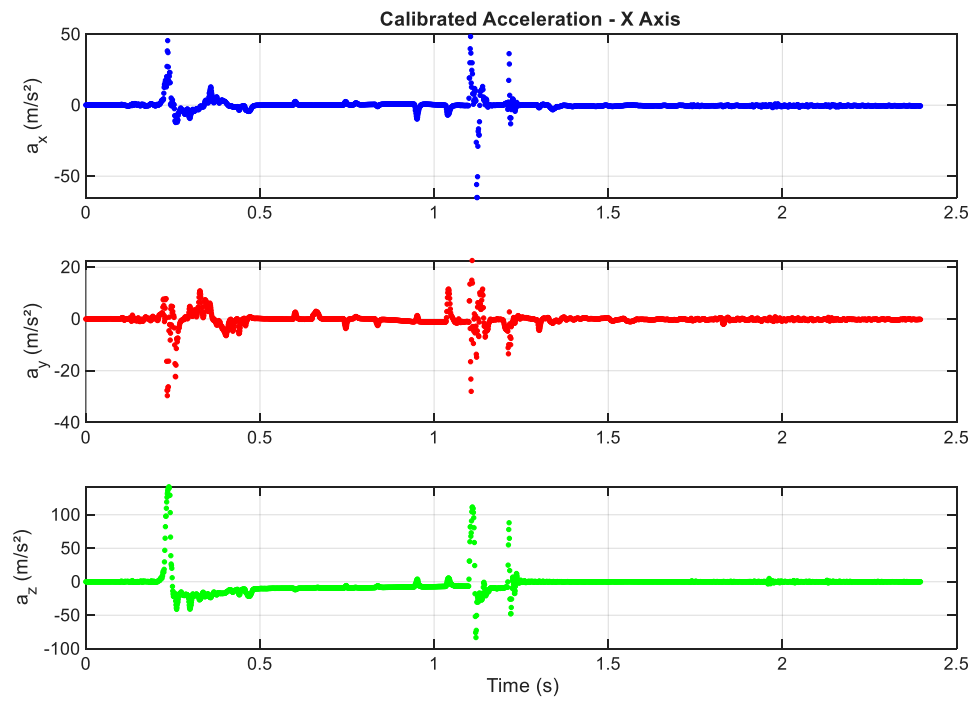
Cal – Position - 400g

Raw Acceleration Data



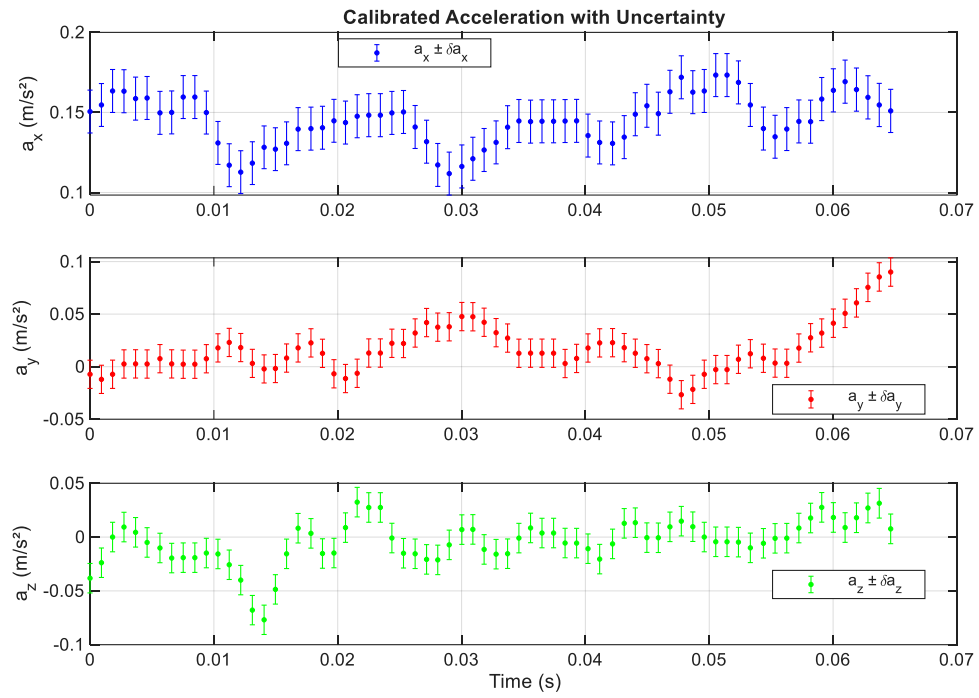
Raw -16g

Calibrated Acceleration Data



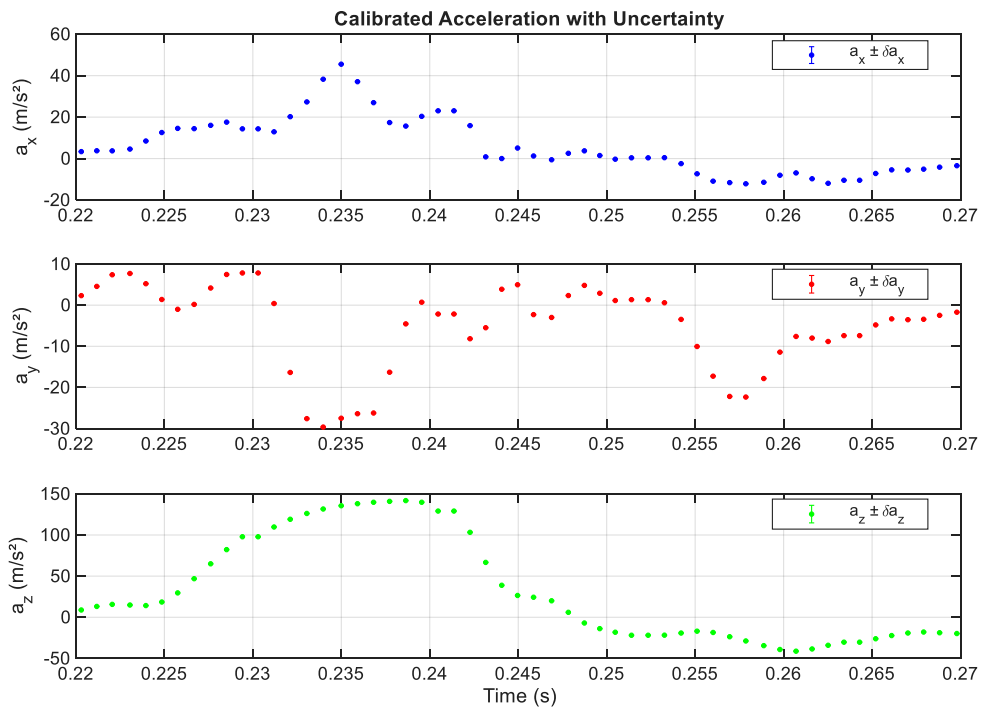
Cal -16g

Calibrated Acceleration with Error Bars [0.000 – 0.065] s



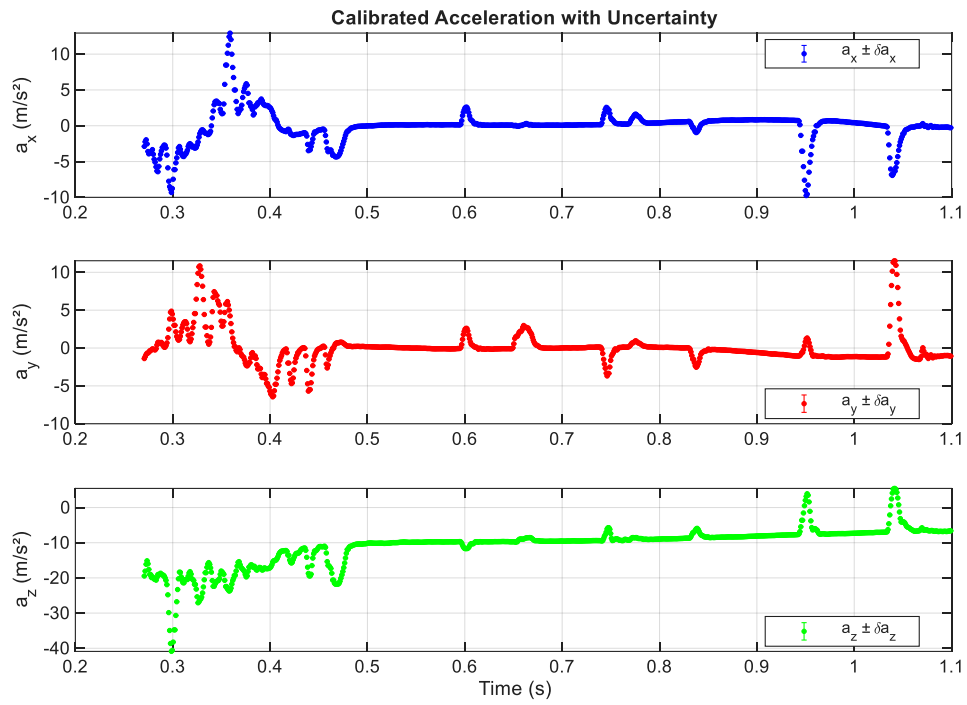
Cal - 16g – Rest

Calibrated Acceleration with Error Bars [0.220 – 0.270] s



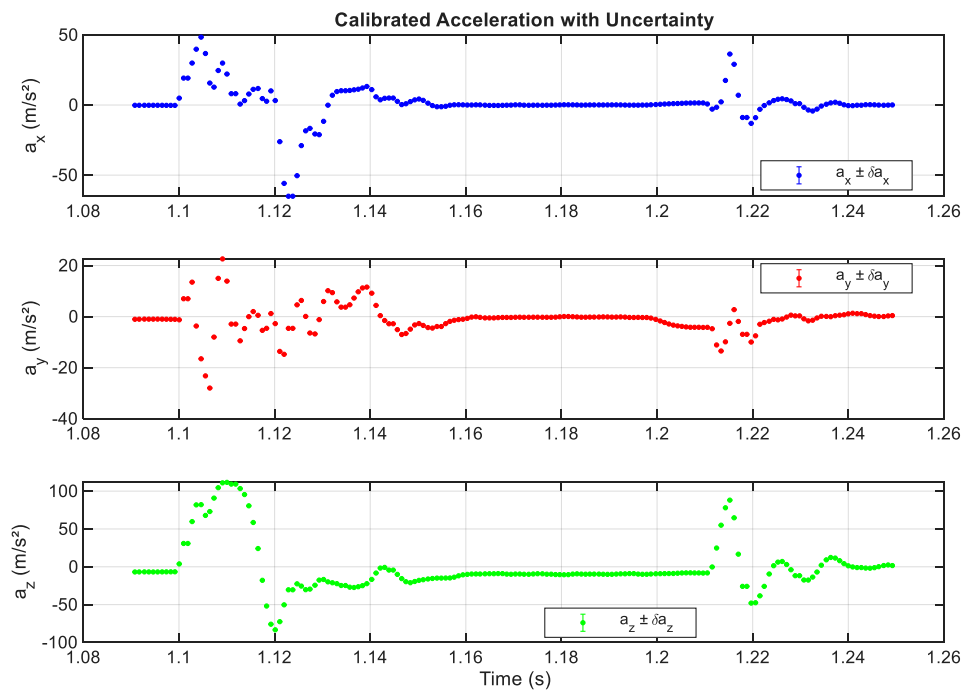
Cal – 16g – Boost & Burn

Calibrated Acceleration with Error Bars [0.270 – 1.100] s



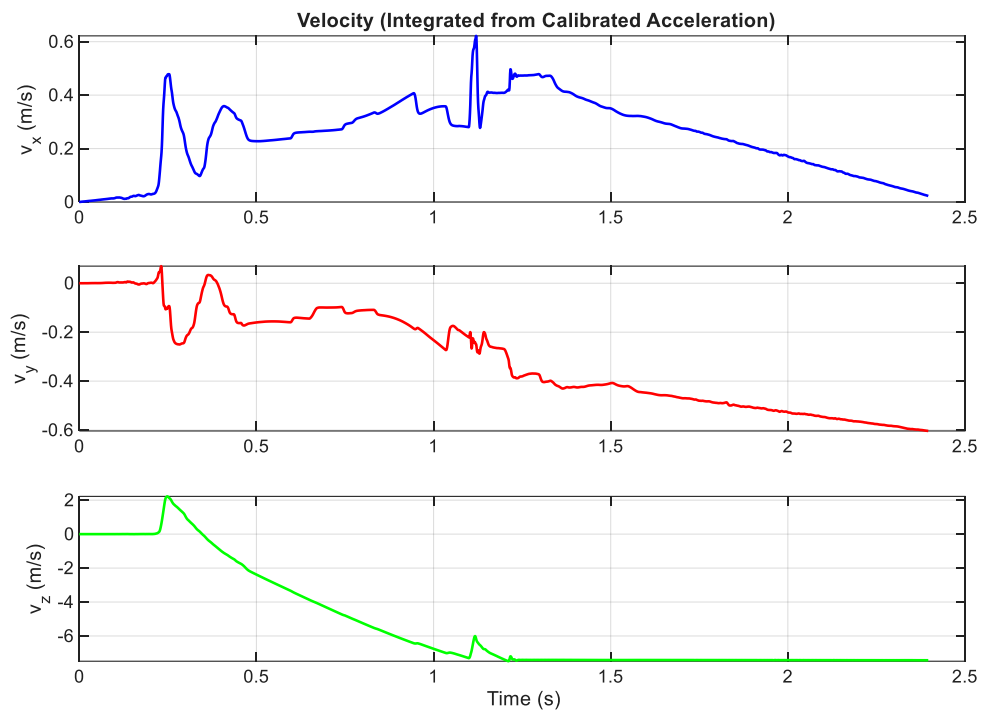
Cal – 16g -Coast

Calibrated Acceleration with Error Bars [1.090 – 1.250] s



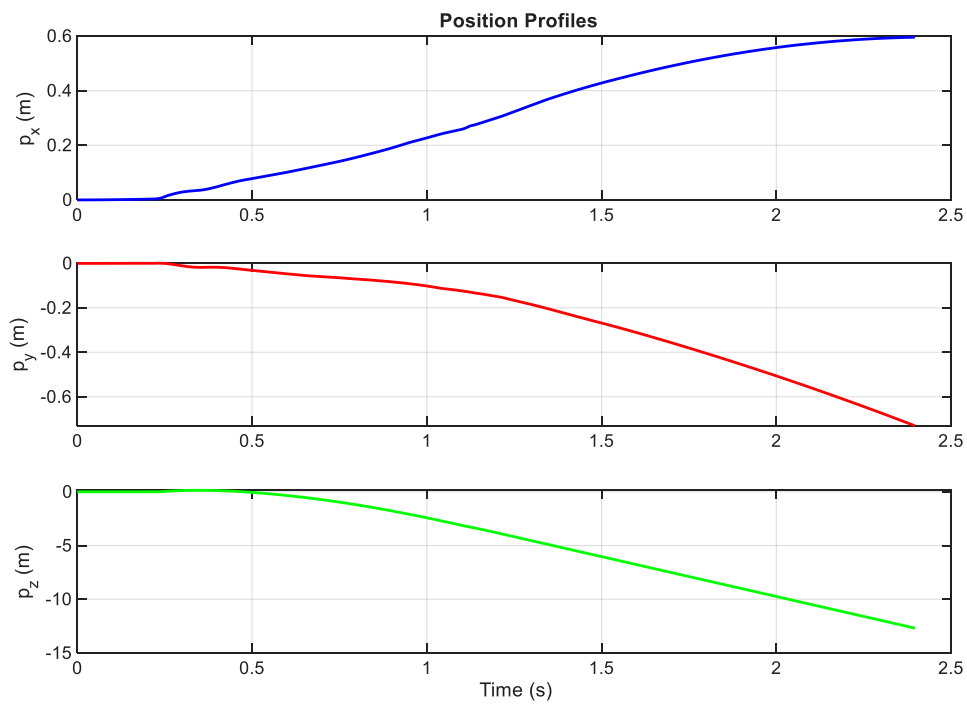
Cal – 16g – Crash

Velocity Profiles



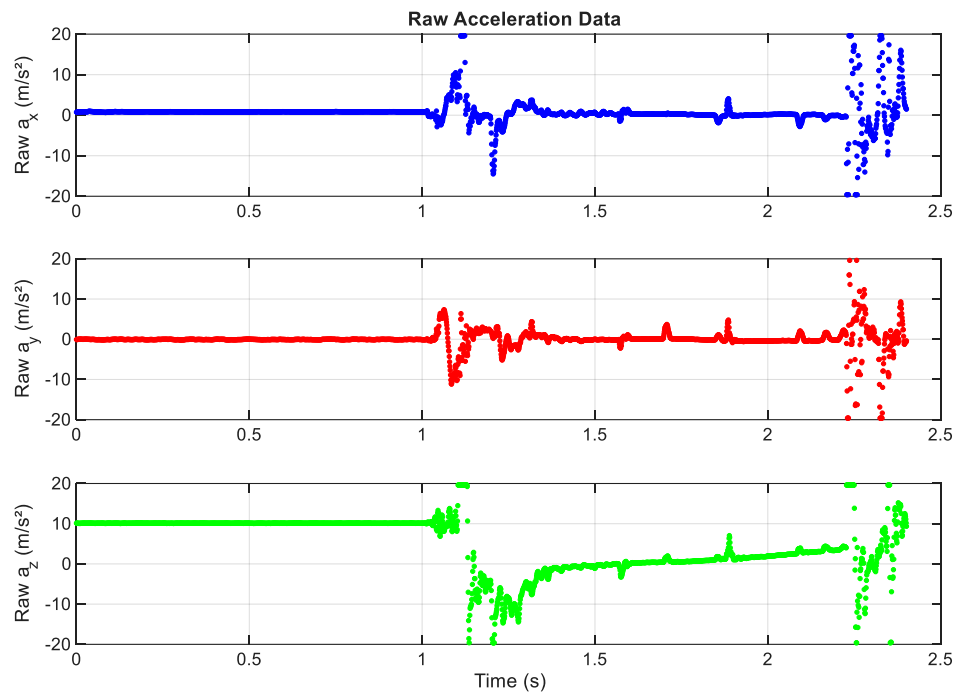
Cal – Velocity – 16g

Position (Double Integrated from Calibrated Acceleration)



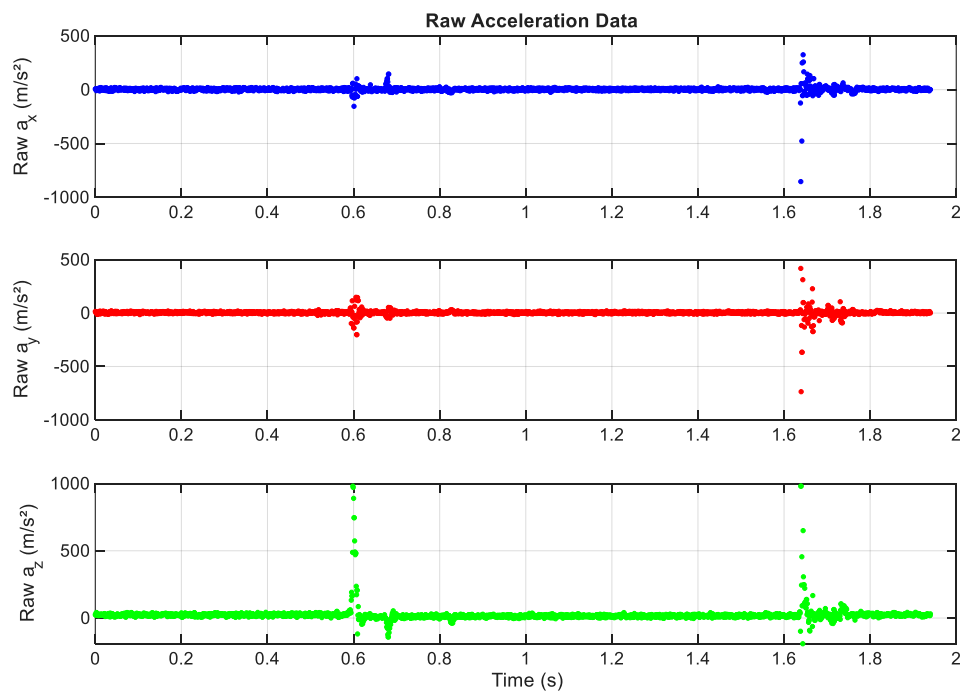
Cal – Position – 16g

Raw Acceleration Data



Raw - 2g

Raw Acceleration Data



Raw -100g

CODE

Microcontroller (C++) – Two synchronous scripts (see **** delineation)

```
#ifndef defines_h
#define defines_h
#ifndef defined(ARDUINO_RASPBERRY_PI_PICO_W)
#error For RASPBERRY_PI_PICO_W only
#endif

#include <WiFi.h>

char ssid[] = "cuWaterRouter"; // your network SSID (name)
char pass[] = "mimmoop2026!"; // your network password (use for WPA, or use as key for WEP), length must be 8+

#endif //defines_h

*****

#include "defines.h"

#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>
#include <Adafruit_H3LIS331.h>
#include <AsyncUDP_RP2040W.h>

// SENSORS + I2C

Adafruit_MPU6050 mpu;
Adafruit_H3LIS331 lis;

TwoWire CustomI2C0(i2c0, 20, 21);

// WIFI / UDP

IPAddress pcIP(192, 168, 0, 100);

#define MSG_PORT 4444

AsyncUDP Udp;

int status = WL_IDLE_STATUS;

// =====
// CSV SEND FUNCTIONS
// =====
// -----
// -----
void sendLowGCSV(
uint32_t us,
float ax,
float ay,
float az
int len = snprintf()
{
char buf[96];
int len = snprintf( char buf[96];

buf, int len = snprintf(

buf, int len = snprintf(
buf,
sizeof(buf),
"%u,%f,%f,%f\n",
us,
ax,
ay,
az
);

Udp.write((uint8_t*)buf, len);
}

// HIGH-G CSV// -----
// HIGH-G CSV
// -----

void sendHighGCSV(
uint32_t us,
float hx,
float hx,
float hz
)
{
char buf[96];

int len = snprintf(
buf),
sizeof(buf),
"%u,%f,%f,%f\n",
us,
hx,
hy,
hz
// ===== hz
);

Udp.write((uint8_t*)buf, len);
}

// =====
```

```

// CORE 0 — SENSOR ACQUISITION
// =====

void setup()
{
  Serial.begin(115200);

  while (!Serial)
    delay(10);

  Serial.println("Core0 init");

  // -----
  // I2C
  // -----
  CustomI2C0.begin();

  // 1 MHz I2C
  CustomI2C0.setClock(1000000);

  // MPU6050 // -----
  // MPU6050
  // -----

  { if (!mpu.begin(MPU6050_I2CADDR_DEFAULT, &CustomI2C0, 0))
    {
      Serial.println("MPU6050 failed");

      while (1)
        tight_loop_contents();
    }

    // -----
    // H3LIS331
    // -----

    if (!lis.begin_I2C(0x18, &CustomI2C0, 0))
    {
      Serial.println("H3LIS331 failed");

      while (1)
        tight_loop_contents();
    }
    // ===== }

    // =====
    // LOW-G SETTINGS (MPU6050)
    // =====

    mpu.setAccelerometerRange(MPU6050_RANGE_16_G);

    mpu.setGyroRange(MPU6050_RANGE_250_DEG);

    mpu.setFilterBandwidth(MPU6050_BAND_44_HZ);

    // =====
    // HIGH-G SETTINGS (H3LIS331)
    // =====

    lis.setRange(H3LIS331_RANGE_400_G);

    lis.setDataRate(LIS331_DATARATE_1000_HZ);

    Serial.println("Core0 ready");
  }

  // =====
  // MAIN LOOP
  // =====

  void loop()
  // ===== {
  uint32_t us = micros();

  // =====
  // HIGH-G ACQUISITION + TRANSMISSION
  // =====

  sensors_event_t highGEvent;

  lis.getEvent(&highGEvent);

  float hx = highGEvent.acceleration.x;
  float hx = highGEvent.acceleration.x;
  float hx = highGEvent.acceleration.x;

  sendHighGCSV(us, hx, hy, hz);

  // =====
  // LOW-G ACQUISITION + TRANSMISSION
  // =====
  /*
  sensors_event_t a, g, t;

  mpu.getEvent(&a, &g, &t);

  float ax = a.acceleration.x;
  float ax = a.acceleration.x;
  float az = a.acceleration.z;

  sendLowGCSV(us, ax, ay, az);
  */
}

// =====
// CORE 1 — WIFI + UDP
// =====

void setup1()
{
  Serial.println("Core1 init");

  if (WiFi.status() == WL_NO_MODULE)

```

```

{
  Serial.println("WiFi module failed");

  while (1)
    tight_loop_contents();
  while (status != WL_CONNECTED) }

  status = WiFi.begin(ssid, pass);

  while (status != WL_CONNECTED)
  {
    delay(200);

    status = WiFi.status();
  }

  Serial.println("WiFi connected");

  if (Udp.connect(IP, MSG_PORT))
    Serial.println("UDP connected");
  else
    Serial.println("UDP failed");
}

void loop()
{
  // idle core
  delay(1);
}
#ifdef defines_h
#define defines_h

#ifndef defined(ARDUINO_RASPBERRY_PI_PICO_W) )
#error For RASPBERRY_PI_PICO_W only
#endif

#include <WiFi.h>

#include <WiFi.h>

#include <WiFi.h>

char ssid[] = "cuWaterRouter"; // your network SSID (name)
char pass[] = "mmmoop2026!"; // your network password (use for WPA, or use as key for WEP), length must be 8+

#endif //defines_h

```

Calibration (Python)

```

import socket
import time
import numpy as np

import numpy as np
UDP_PORT = 4444

CALIB_TIME_SEC = 5.0

def main():
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.bind((UDP_IP, UDP_PORT))

    print("Calibration: keep sensor stationary")

    samples = []
    start = time.time()

    # ----- collect calibration data -----
    while time.time() - start < CALIB_TIME_SEC:
        data, _ = sock.recvfrom(1024)

        try:
            line = data.decode().strip()

        try:
            line = data.decode().strip()
            parts = line.split(",")
            if len(parts) != 4:
                continue

            _, ax, ay, az = parts

        except:
            continue

    sock.close()

    if len(samples) == 0:
        print("No data received.")
        return

    samples = np.array(samples)
    samples = np.array(samples)

    # ----- compute bias (gravity included) -----
    bias = np.mean(samples, axis=0)

    print("\nCalibration complete")
    print("Bias vector (ax, ay, az):")
    print(f"({bias[0]:.6f}, {bias[1]:.6f}, {bias[2]:.6f})")

    print("\nProgram terminated.")

    main()if __name__ == "__main__":
    main()

```

Data to CSV to Excel (Python)

```
import socket
import csv
import os
from datetime import datetime
UDP_IP = "0.0.0.0"
UDP_PORT = 4444

CSV_PATH = r"C:\Users\jacob\data\accelerometer_data.csv"

def main():
    print("Writing CSV to:", CSV_PATH)

    # Allow socket reuse (prevents WinError 10048 on restart)
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

    sock.bind((UDP_IP, UDP_PORT))

    # Ensure folder exists
    os.makedirs(os.path.dirname(CSV_PATH), exist_ok=True)

    with open(CSV_PATH, "w", newline="") as f:
        writer = csv.writer(f)

        # CSV header
        writer.writerow(["pc_time", "pico_us", "ax", "ay", "az"])
        f.flush()
        os.fsync(f.fileno())

    print(f"Listening on UDP {UDP_IP}:{UDP_PORT}\n")

    while True:
        data, addr = sock.recvfrom(1024)
        try:
            text = data.decode("utf-8").strip()
        except UnicodeDecodeError:
            print("BAD PACKET (non-utf8).", data)
            continue

        # handle possible multiple lines per UDP packet
        for line in text.split("\n"):
            parts = line.split(",")

            if len(parts) != 4:
                print("SKIP:", repr(line))
                continue

            pico_us, ax, ay, az = parts

            pc_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]

            row = [pc_time, pico_us, ax, ay, az]

            print(row)

            writer.writerow(row)

            # force write to disk immediately
            f.flush()
            os.fsync(f.fileno())

if __name__ == "__main__":
    main()
```

Post Processing (MATLAB)

```
clc; close all;

%% Calibration Accelerations
% Low g
a1p = [ 9.975113, 0.396280, -0.184676 ]; % +x
a1n = [-9.533770, -0.107105, 0.481328 ]; % -x
%
a2p = [ -0.002453, 9.769963, 0.324655 ]; % +y
a2n = [ 0.507053, -9.740908, 0.031087 ]; % -y
%
a3p = [ 0.636985, -0.050985, 10.124125 ]; % +z
a3n = [ -0.208125, 0.143905, -9.669601 ]; % -z

% High g
a1p = [ 18.262427, 8.755777, 0.653105 ]; % +x
a1n = [ -1.226269, 10.014580, 0.829213 ]; % -x

a2p = [ 8.806548, 20.173581, 1.050405 ]; % +y
a2n = [ 8.665104, 0.285398, 0.406510 ]; % -y

a3p = [ 8.542123, 9.615942, 10.084436 ]; % +z
a3n = [ 8.328015, 10.134971, -8.294616 ]; % -z

g = 9.80;

%% The Bias Vector
bx = ((a1p(1) + a1n(1))/2 + (a2p(1) + a2n(1))/2 + (a3p(1) + a3n(1))/2)/3;
by = ((a1p(2) + a1n(2))/2 + (a2p(2) + a2n(2))/2 + (a3p(2) + a3n(2))/2)/3;
bz = ((a1p(3) + a1n(3))/2 + (a2p(3) + a2n(3))/2 + (a3p(3) + a3n(3))/2)/3;

b = [bx; by; bz];

fprintf('Bias Vector:\n');
disp(b);
```

```

% Remove Bias
a1 = a1p - b;
a2 = a2p - b;
a3 = a3p - b;

A = [a1 a2 a3]; % Unbiased Matrix

%% Calibration Matrix
C = g * inv(A);

fprintf('Calibration Matrix:\n');
disp(C);

S = det(C);

fprintf('Calibration Matrix Determinant:\n');
disp(S);

%% Import Raw Data
pico_us = accelerometer_data.pico_us';

ax = accelerometer_data.ax';
ay = accelerometer_data.ay';
az = accelerometer_data.az';

T = 1e-6 * (pico_us - pico_us(1)); % Time Vector
Araw = [ax; ay; az]; % Data Matrix

%% Apply Calibration
Acal = C * (Araw - b);

% Remove Static Gravity
A_motion = Acal - [0;0;g];

% Calibrated Components
axcal = A_motion(1,:);
aycal = A_motion(2,:);
azcal = A_motion(3,:);

%% Calibrated Accelerations - Separate Plots
figure('Position', [100 100 900 700]);
set(gcf, 'WindowStyle', 'docked'); % Auto-dock

subplot(3,1,1);
plot(T, axcal, 'o', 'MarkerSize', 2, 'LineStyle', 'none', ...
     'MarkerFaceColor', 'b', 'MarkerEdgeColor', 'b');
ylabel('a_x (m/s^2)'); title('Calibrated Acceleration - X Axis'); grid on;

subplot(3,1,2);
plot(T, aycal, 'o', 'MarkerSize', 2, 'LineStyle', 'none', ...
     'MarkerFaceColor', 'r', 'MarkerEdgeColor', 'r');
ylabel('a_y (m/s^2)'); grid on;

subplot(3,1,3);
plot(T, azcal, 'o', 'MarkerSize', 2, 'LineStyle', 'none', ...
     'MarkerFaceColor', 'g', 'MarkerEdgeColor', 'g');
xlabel('Time (s)'); ylabel('a_z (m/s^2)'); grid on;

sgtitle('Calibrated Acceleration Data');

%% Raw (Uncalibrated) Accelerations - Separate Plots
figure('Position', [1050 100 900 700]);
set(gcf, 'WindowStyle', 'docked'); % Auto-dock

subplot(3,1,1);
plot(T, Araw(1,:), 'o', 'MarkerSize', 2, 'LineStyle', 'none', ...
     'MarkerFaceColor', 'b', 'MarkerEdgeColor', 'b');
ylabel('Raw a_x (m/s^2)'); title('Raw Acceleration Data'); grid on;

subplot(3,1,2);
plot(T, Araw(2,:), 'o', 'MarkerSize', 2, 'LineStyle', 'none', ...
     'MarkerFaceColor', 'r', 'MarkerEdgeColor', 'r');
ylabel('Raw a_y (m/s^2)'); grid on;

subplot(3,1,3);
plot(T, Araw(3,:), 'o', 'MarkerSize', 2, 'LineStyle', 'none', ...
     'MarkerFaceColor', 'g', 'MarkerEdgeColor', 'g');
xlabel('Time (s)'); ylabel('Raw a_z (m/s^2)'); grid on;

sgtitle('Raw Acceleration Data');

%% Uncertainty Analysis
t0 = 0.27;
t1 = 1.1;

% Uncertainties
delta_16g = .00479;
delta_400g = 1.91;

% Uncertainty Input
delta_a_tilde = delta_400g;

idx = T >= t0 & T <= t1;
t_plot = T(idx);

a_mat = A_motion(:, idx);
inner = 2*[1;1;1] + (1/g) * sum(a_mat, 1);
delta_a_vec = (1 + 1/sqrt(6)) * delta_a_tilde * (C * inner);

delta_ax = abs(delta_a_vec(1,:));
delta_ay = abs(delta_a_vec(2,:));
delta_az = abs(delta_a_vec(3,:));

%% Plot - X, Y, Z with Error Bars
figure('Position', [150 100 950 700]);
set(gcf, 'WindowStyle', 'docked'); % Auto-dock

% X Axis - Blue
subplot(3,1,1);
errorbar(t_plot, axcal(idx), delta_ax, 'b', 'MarkerSize', 6, 'CapSize', 2, 'LineStyle', 'none');

```

```

ylabel('a_x (m/s^2)');
title('Calibrated Acceleration with Uncertainty');
grid on;
legend('a_x ± \delta a_x', 'Location','best');

% Y Axis - Red
subplot(3,1,2);
errorbar(t_plot, aycal(idx), delta_ay, 'r', 'MarkerSize',6, 'CapSize',2, 'LineStyle','none');
ylabel('a_y (m/s^2)');
grid on;
legend('a_y ± \delta a_y', 'Location','best');

% Z Axis - Green
subplot(3,1,3);
errorbar(t_plot, azcal(idx), delta_az, 'g', 'MarkerSize',6, 'CapSize',2, 'LineStyle','none');
xlabel('Time (s)');
ylabel('a_z (m/s^2)');
grid on;
legend('a_z ± \delta a_z', 'Location','best');

sgtitle(sprintf('Calibrated Acceleration with Error Bars [%0.3f - %0.3f] s', t0, t1));

% Integrate acceleration → velocity (v(0) = 0)
vx = cumtrapz(T, axcal);
vy = cumtrapz(T, aycal);
vz = cumtrapz(T, azcal);

% Integrate velocity → position (x(0) = 0)
px = cumtrapz(T, vx);
py = cumtrapz(T, vy);
pz = cumtrapz(T, vz);

%% Velocity Figure
figure('Position', [200 100 950 700]);
set(gcf, 'WindowStyle', 'docked');

subplot(3,1,1); plot(T, vx, 'b-', 'LineWidth', 1.3); ylabel('v_x (m/s)'); title('Velocity Profiles'); grid on;
subplot(3,1,2); plot(T, vy, 'r-', 'LineWidth', 1.3); ylabel('v_y (m/s)'); grid on;
subplot(3,1,3); plot(T, vz, 'g-', 'LineWidth', 1.3); xlabel('Time (s)'); ylabel('v_z (m/s)'); grid on;
sgtitle('Velocity (Integrated from Calibrated Acceleration)');

%% Position Figure
figure('Position', [250 100 950 700]);
set(gcf, 'WindowStyle', 'docked');

subplot(3,1,1); plot(T, px, 'b-', 'LineWidth', 1.3); ylabel('p_x (m)'); title('Position Profiles'); grid on;
subplot(3,1,2); plot(T, py, 'r-', 'LineWidth', 1.3); ylabel('p_y (m)'); grid on;
subplot(3,1,3); plot(T, pz, 'g-', 'LineWidth', 1.3); xlabel('Time (s)'); ylabel('p_z (m)'); grid on;
sgtitle('Position (Double Integrated from Calibrated Acceleration)');

%% Data Grab
axcalT = axcal;
aycalT = aycal;
azcalT = azcal;
% copygraphics(figure(3), 'Resolution', 300)

```